



Use cases and use case diagrams

Ingegneria del Software

2025/2026

Università di Trento

Chiara Di Francescomarino

Agenda



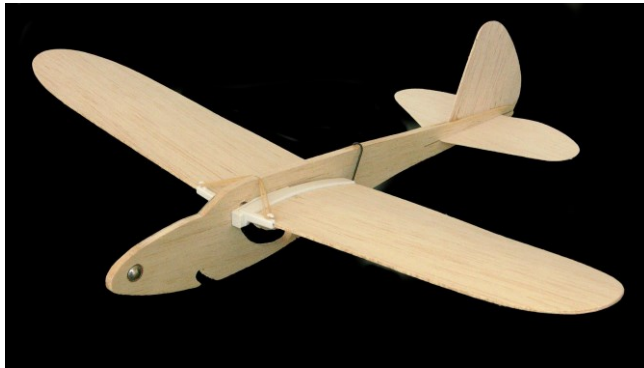
1.

2.

3.

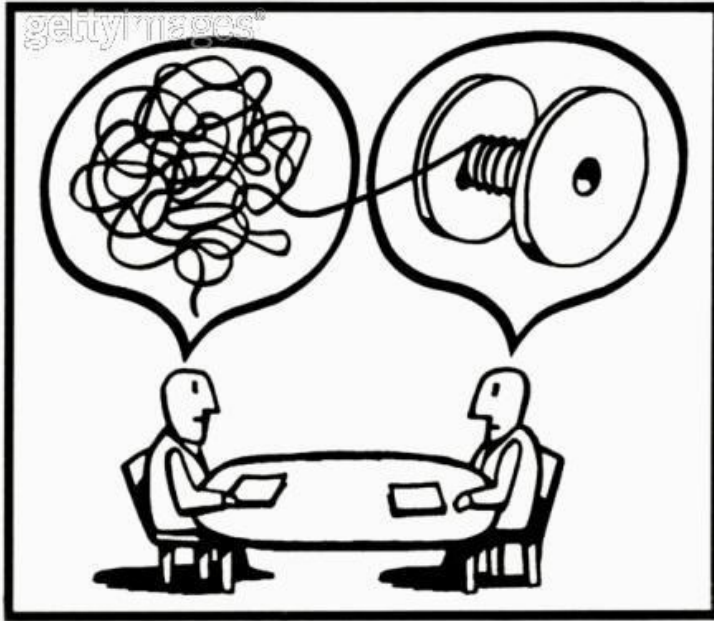
Today

- Models
- Modelling languages
- System modelling perspectives
- Use cases and use case diagrams
 - Use case boundaries
 - Actors
 - Use cases and scenarios
 - Actor – use case associations
- Actor generalizations



Models

The importance of conceptual models



“The activity of formally describing **some** aspects of the **physical** and **social** world around us for the purposes of **understanding** and **communication**.”

(John Mylopoulos, 1992)

Model

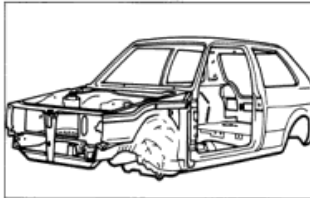


- “A **model** is a simplifying mapping of reality to serve a specific **purpose**”
(Stachowiak: Allgemeine Modelltheorie, 1973)

What are and why do we need models?



Models are **abstractions from real world phenomena**, developed for the purpose of **reducing overall complexity**.



Models **aggregate information** and **document only relevant aspects** of the real world



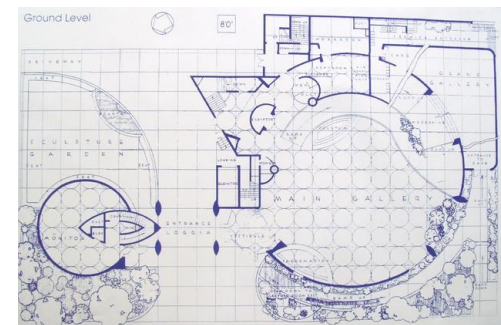
Models are developed:

1. for a specific **target audience**
2. with a specific **modelling purpose** in mind

What's the relevant model?



?



Agenda



1.

2.

3.

Today

- Models
- **Modelling languages**
- System modelling perspectives
- Use cases and use case diagrams
 - Use case boundaries
 - Actors
 - Use cases and scenarios
 - Actor – use case associations
- Actor generalizations

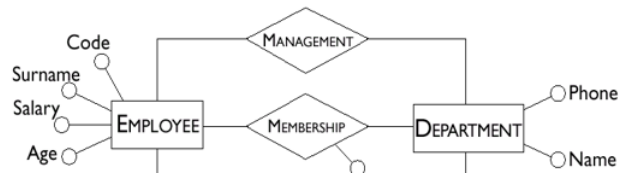


Modelling languages

Choice of the modelling language

- Easy to use
 - For writing the model
 - For reading the model
- Understandability – also by non-technical stakeholders
 - It should support the communication with users and customers
- Level of formality
 - If the language is formal and precise, it can be used for further analyses

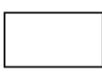
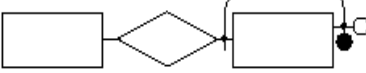
Different languages can be used within the different phases of the software development (different levels of conceptual abstraction) and different purposes



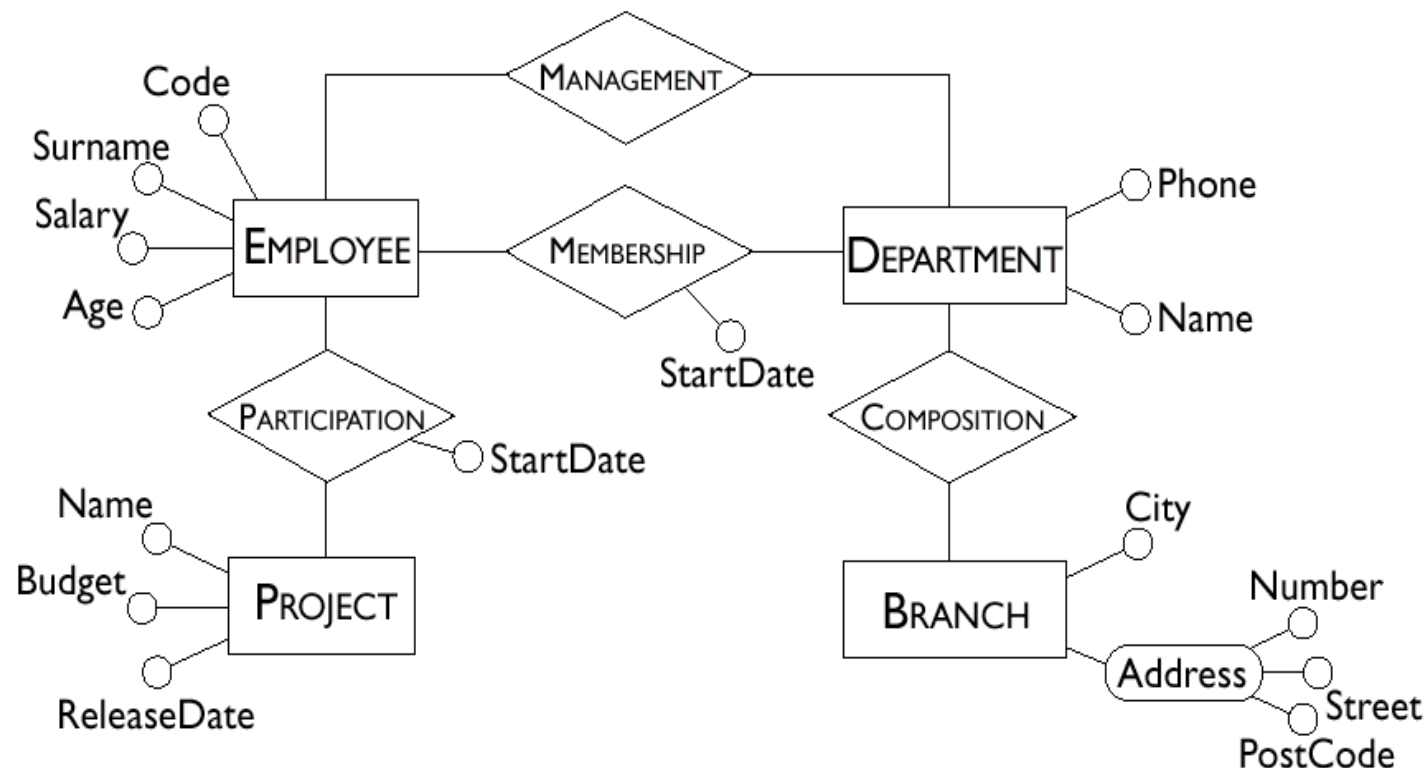
The ER language

The Entity-relationship language

- It is mainly used for the analysis and design of information systems and databases

Construct	Graphical representation
Entity	
Relationship	
Simple attribute	
Composite attribute	
Cardinality of a	
Cardinality of an attribute	
Internal identifier	 
External identifier	
Generalization	
Subset	

The ER model: an example





The UML language

What is UML?

- UML = Unified Modeling Language
- Family of graphic notations useful to support the description and design of software systems
 - In particular OO systems, but not only
- OMG standards
 - Object Management Group
 - Consortium of IT companies
 - Not-for-profit
 - Objective: create standards in the IT context



What UML is not?

- It is not a development method (methodology)!
- It's a language/notation
- It is not a programming language
- It is a modeling language
- It is not tied to any particular methodology
- It can be used in both plan-driven and agile methods

How can we use UML?

- Sketch
 - Purpose: to aid communication and discussion - only most important info (incomplete diagrams)
 - Tool: graphic editor or whiteboard
- Detailed project (blueprint)
 - Purpose: To provide a complete/detailed model to implement so that coding is mainly a mechanical activity
 - Tools: UML modeler
- Towards programming language
 - Purpose: to provide an “executable model”
 - Tools: specific to run/transform the model

A bit of history

80s. Object-oriented languages (e.g., Smalltalk)

1980-1995. "Babel" of modeling languages

1995. Booch and Rumbaugh present at OOPSLA Unified Method v. 0.8. The announcement is made that Rational has bought Objectory where Jacobson works

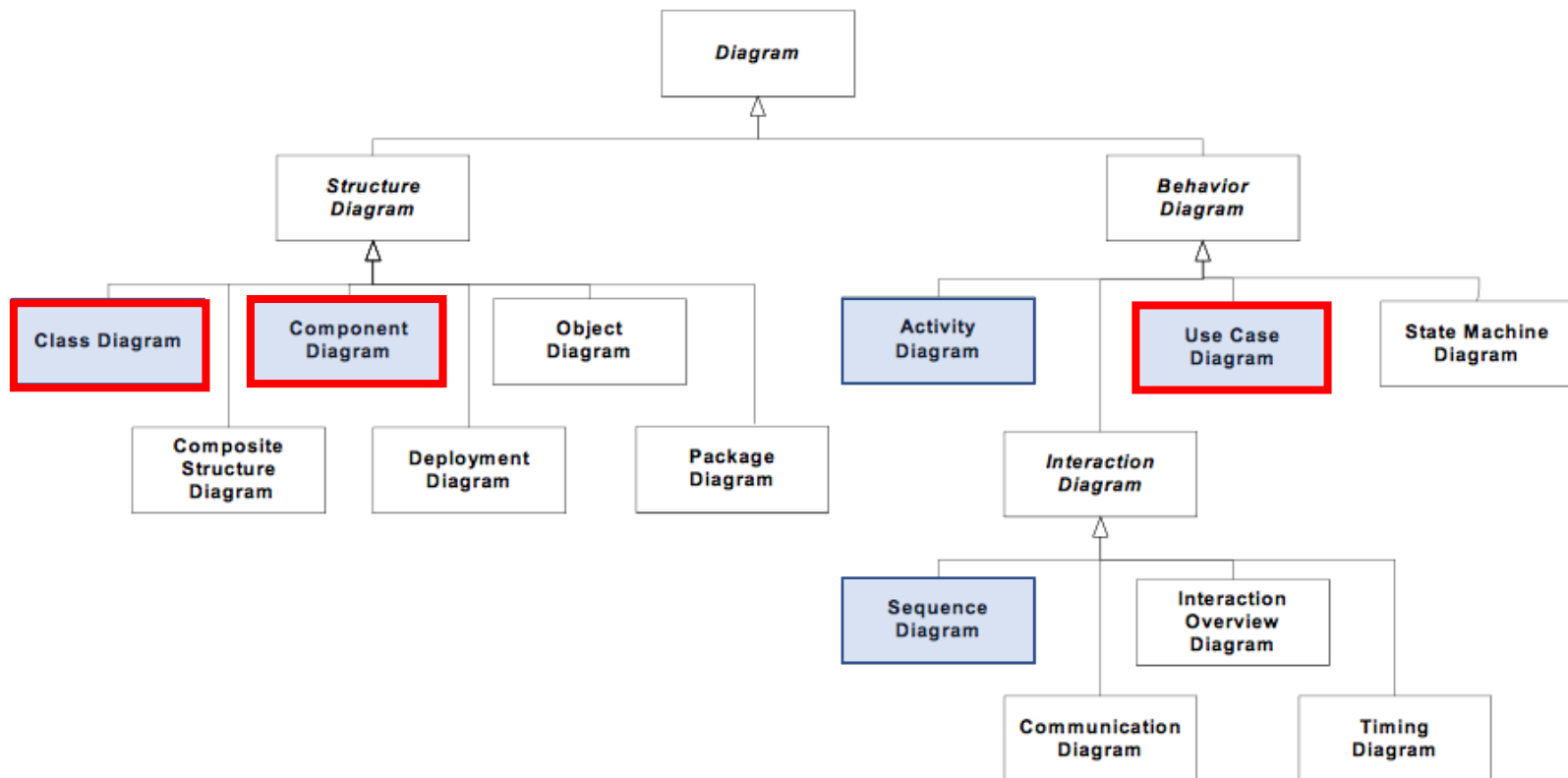
1996-97. Call for proposal from OMG. The "three Amigos" (Booch, Rumbaugh and Jacobson) propose UML 1.0.

2005. OMG propose UML 2.0.

Why UML?

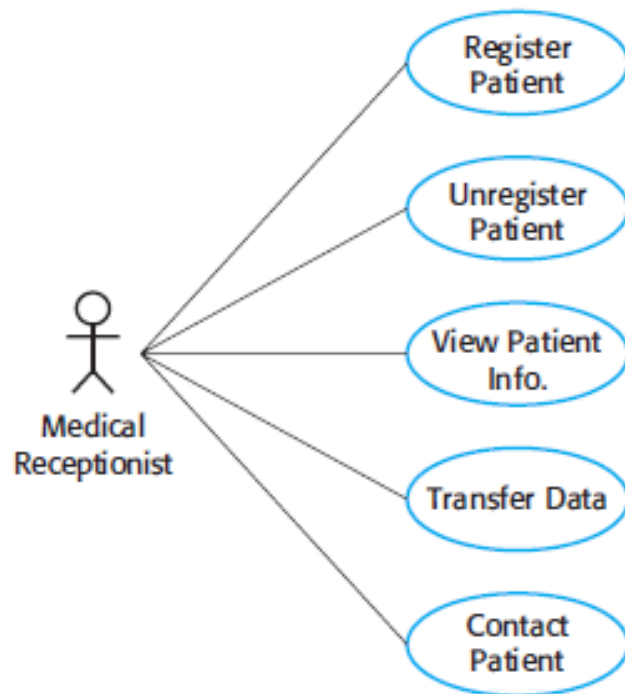
- It's a standard
 - UML plays a very important "lingua franca" function in the ICT world
- Fairly used/requested by the industry
 - At least at the sketch level
 - It is required in some public tenders
- Sometimes a picture (diagram) is better than 1000 words!

UML (Unified Modeling Notation)



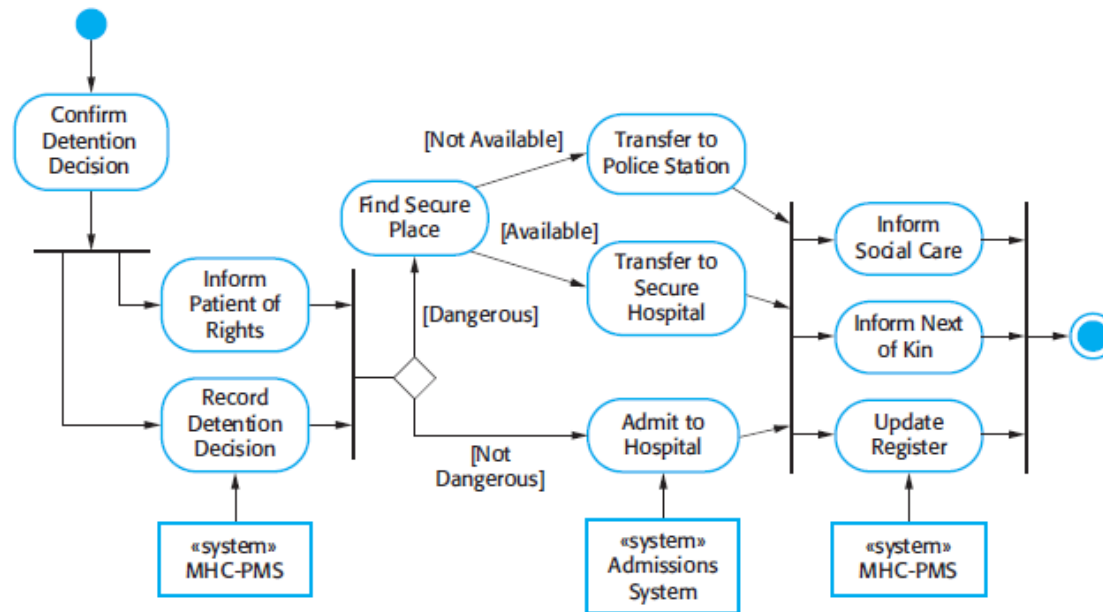
Use case diagrams

Use case diagrams show the interactions between a system and its environment.



Activity diagrams

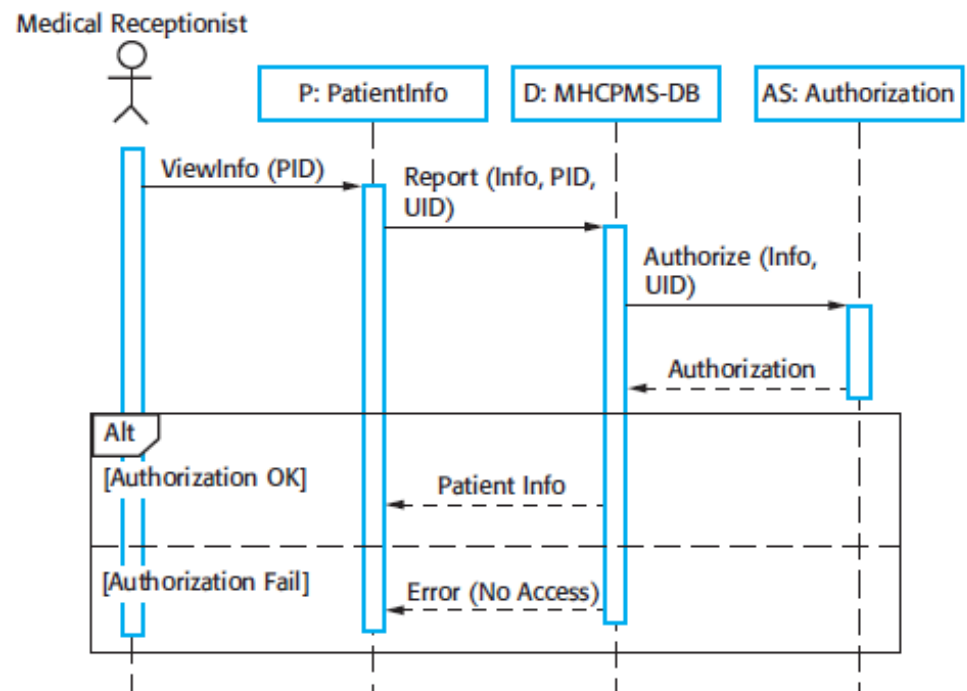
Activity diagrams show the activities involved in a process or in data processing.



We will see also another notation for representing the dynamic behavior

Sequence diagrams

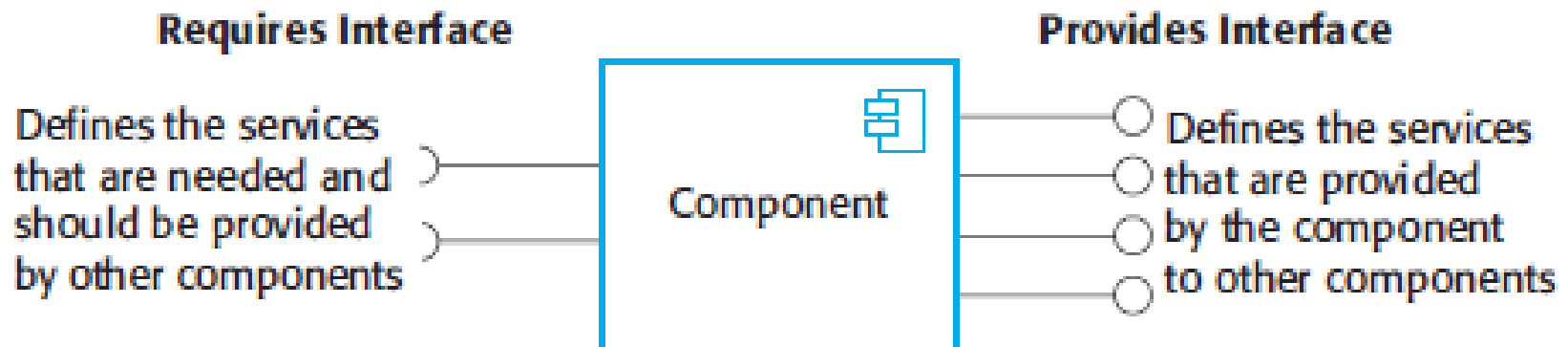
Sequence diagrams show **interactions** between actors and the system and between system components



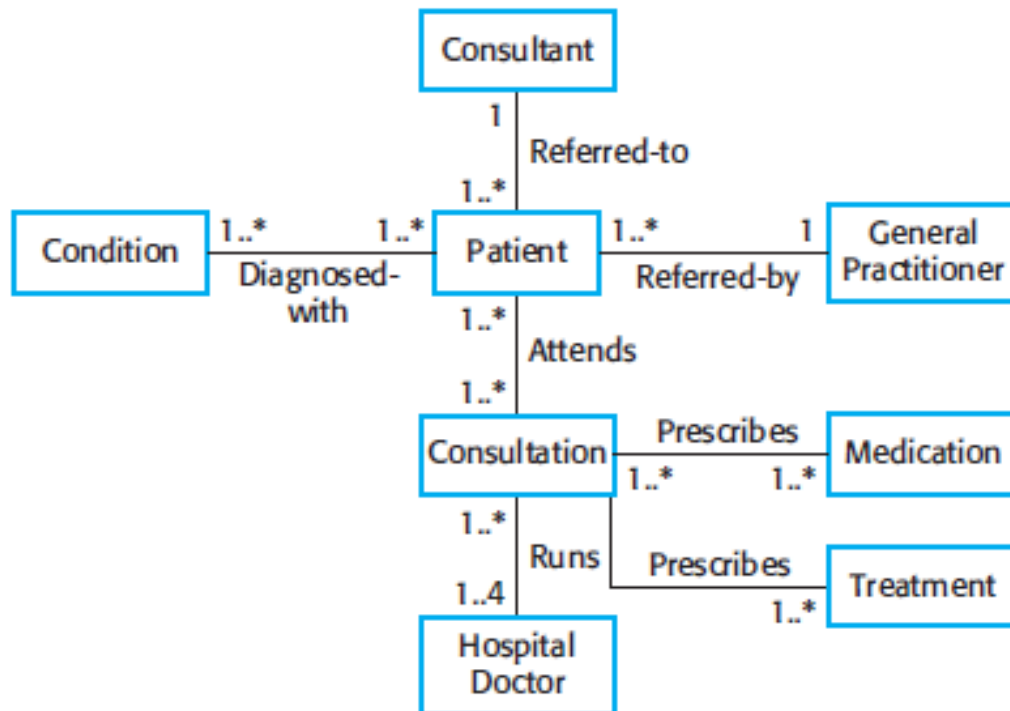
We will see also another notation for representing the dynamic behavior

Component diagrams

- Component diagrams show **software components** and their relationships



Class diagrams



Class diagrams show the **object classes** in the system and the **associations** between these classes.

The OCL Language

- Object Constraint Language
 - Functional language for writing **expressions** and **constraints** on object-oriented models
 - Proposed as an extension of UML (OMG)
- Query language
 - Evaluation of expressions
 - A constraint is a restriction on one or more values of parts of the UML model
 - E.g., for a Boolean variable an OCL expression can be used to check that the value is always true
 - Usage of OCL expressions as notes in the diagram (context)

Let's try a test

Join this Wooclap event



1

Go to wooclap.com

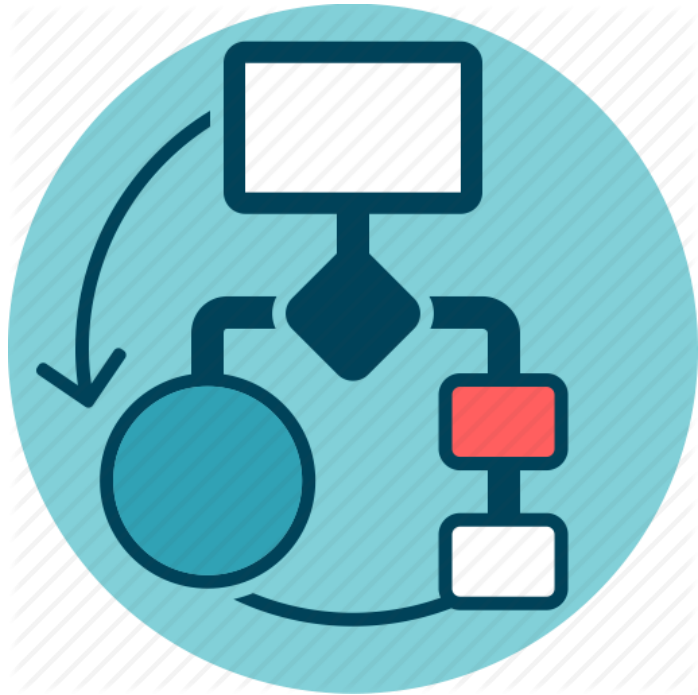
2

Enter the event code in the top banner

Event code

SAHUKX

 [Copy participation link](#)

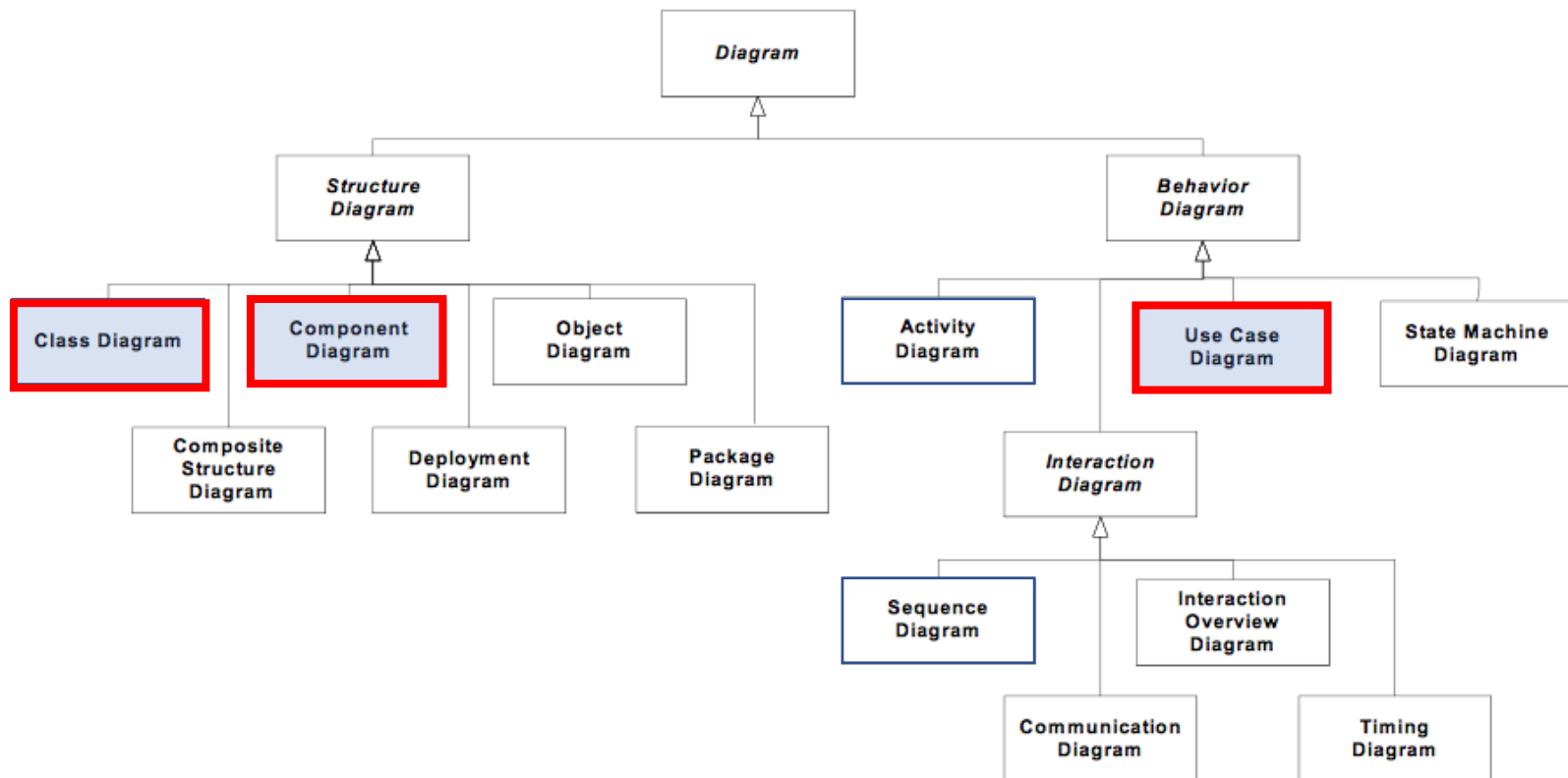


Models and system modeling

Use of graphical models

- As a means of facilitating discussion about an existing or proposed system
 - Incomplete and incorrect models are OK as their role is to support discussion.
- As a way of documenting an existing system
 - Models should be an accurate representation of the system but need not be complete.
- As a detailed system description that can be used to generate a system implementation
 - Models have to be both correct and complete.

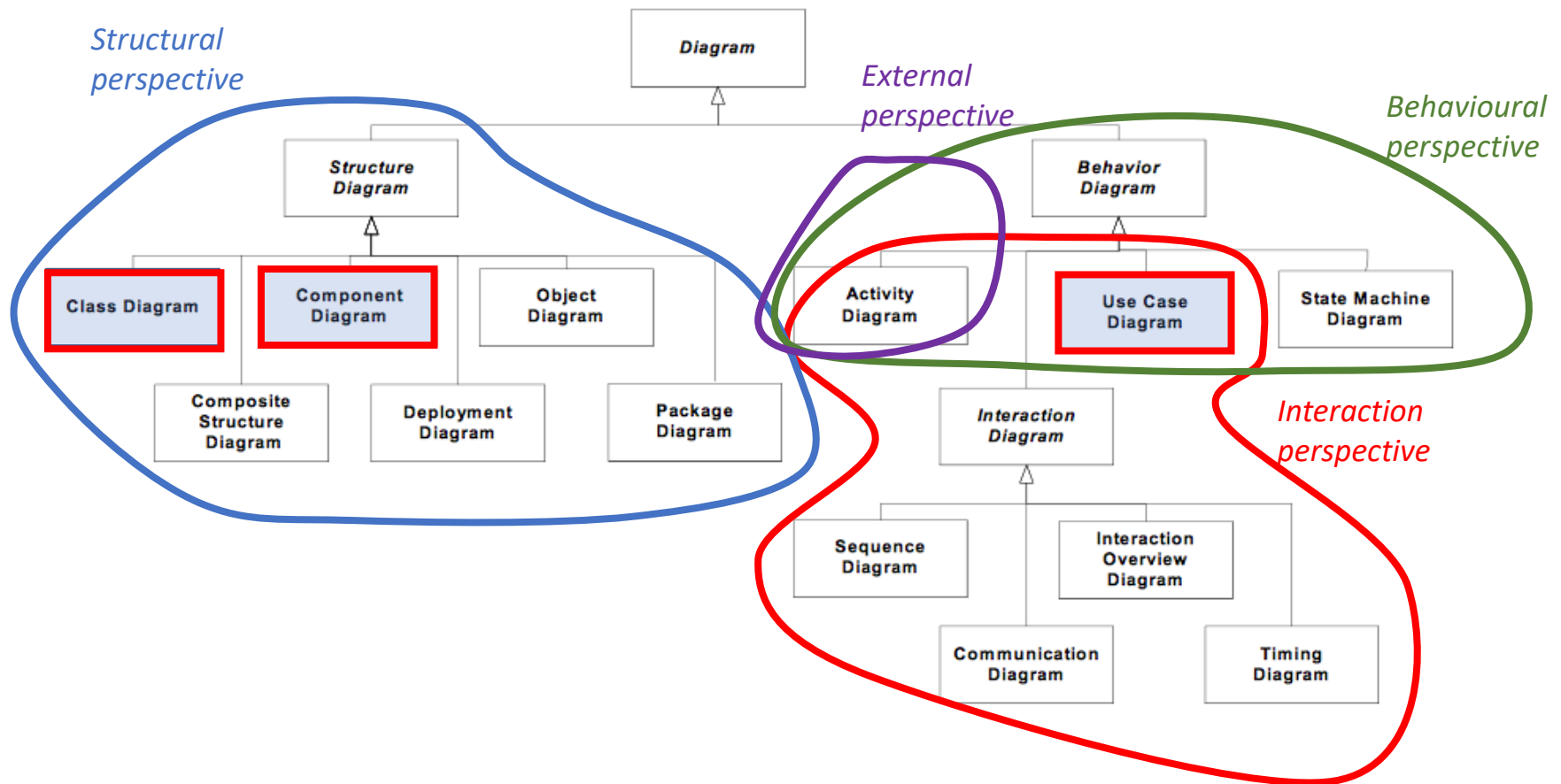
UML (Unified Modeling Notation)



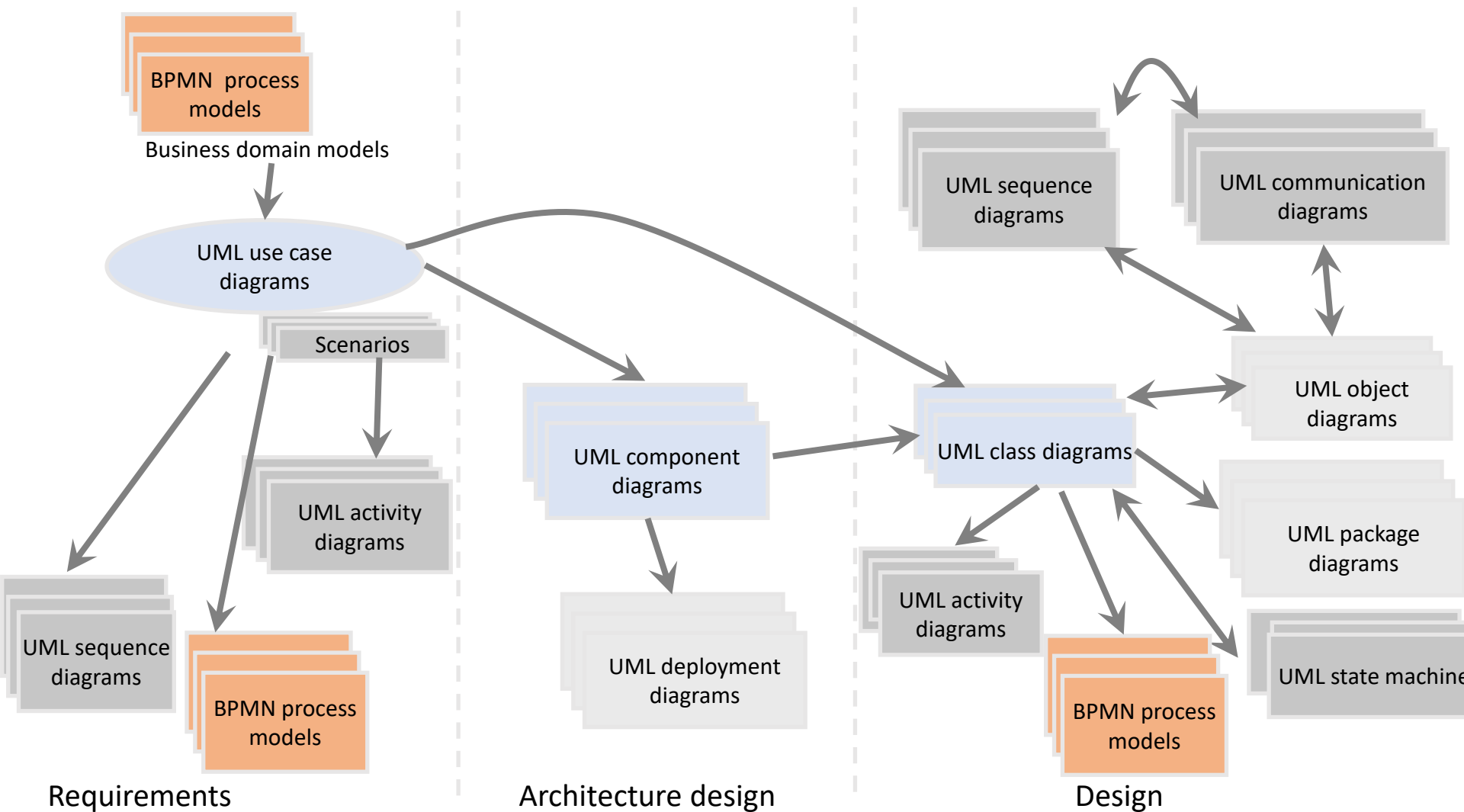
System modelling and perspectives

- System modeling is the process of **developing abstract models** of a system, with each model presenting a different view or perspective of that system.
- Different system perspectives:
 - **Interaction** perspective: interactions between the system and its environment or between the components of a system
 - **Structural** perspective: organization of a system or the structure of the data processed in the system
 - **Behavioral** perspective: dynamic behavior of the system and how it responds to events
 - **Data** Some data arrives that has to be processed by the system.
 - **Events** Some event happens that triggers system processing. Events may have associated data, although this is not always the case.
 - **External** perspective: the context or environment of the system

UML (Unified Modeling Notation)



The overall view



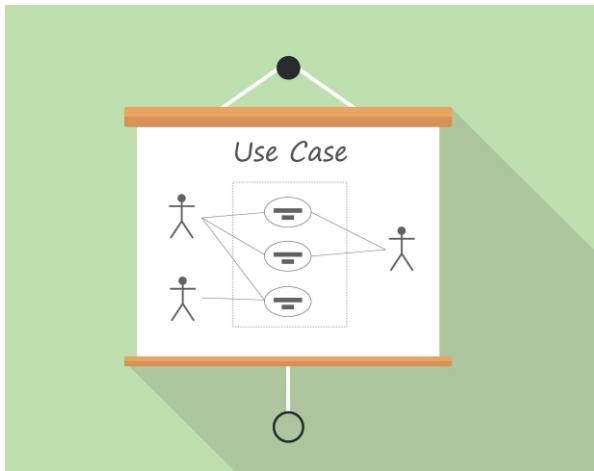
Interaction models

- Interactions can be modeled at different levels of abstraction
 - At a high-level **use cases** describe how a system interacts with outside actors
 - Each use case represents a functionality that a system provides to the user
 - Use cases are helpful for capturing **informal requirements**
 - **Sequence/activity/BPMN diagrams** provide more details about **which operations need to be invoked in a specific scenario**

Why are interaction models important?

- Modeling user interaction is important as it helps to **identify user requirements**.
- Modeling system-to-system interaction highlights the **communication problems** that may arise.
- Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

Use cases and use case diagrams



Use cases

- Proposed by Ivar Jacobson in 1992
 - Developed to support requirements elicitation
-
- Use cases are **operating scenarios** of the system users
 - Scenarios are ways in which the system can be used (they define the operations or functions that the system offers to its users)



Use case

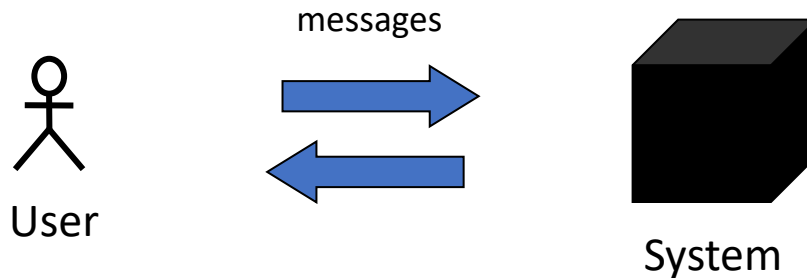
- Each use case represents a discrete task that involves external interaction with a system
- A **step-by-step description** of how a user interacts with the software system to accomplish a business goal
- Used for discovering and documenting the **functional requirements** of a system from the **viewpoint of the user**

Use case

- It represents a **functional requirement**
- It makes clear what you expect from a system (“**what?**”)
- It hides the behaviour of the system (“**how?**”)
- It is a **sequence of actions** (with variants) that produce a result observable by an actor

Use case

- The system is seen as a black-box



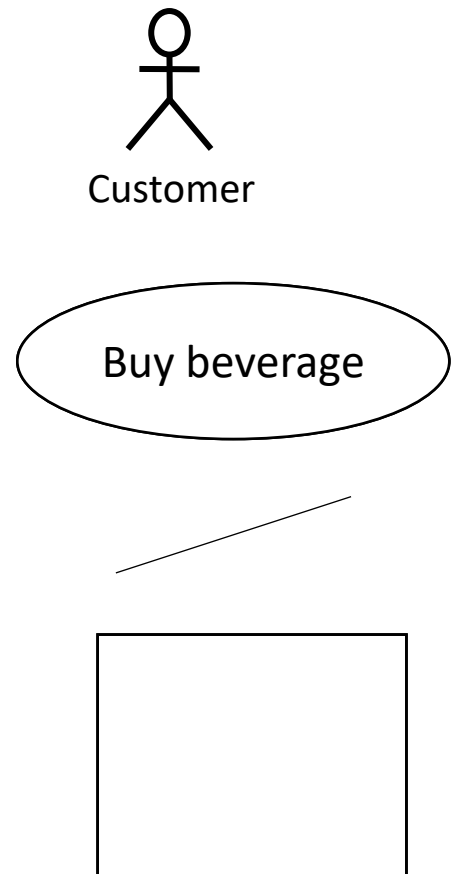
- Use cases express the **interaction** (exchange of messages) between the entities that interact with the system itself (called actors)

Use case diagram

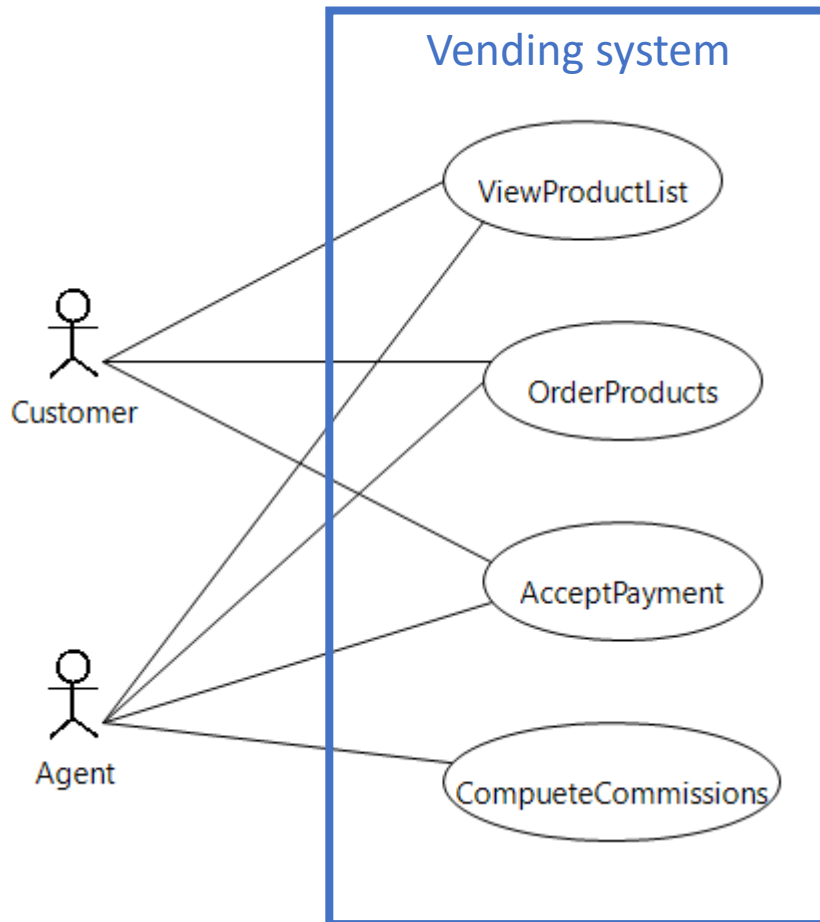
- Specifies:
 - **Actors** → human or automatic actors using the system and its functionalities
 - **Use cases** → functionalities of the system
 - The **relations** between the functionalities and the users
 - **System boundaries**
- Describe the behaviour of the system without specifying how it is going to be implemented

Use case diagram

- **Actors:** roles of the entities (persons, hw/sw systems, devices) that interact with the system (stick man)
- **Use case:** what the actors can do with the system (ellipse with the name inside)
- **Actor-case associations:** relations among actors and use cases (lines)
- **System boundaries**

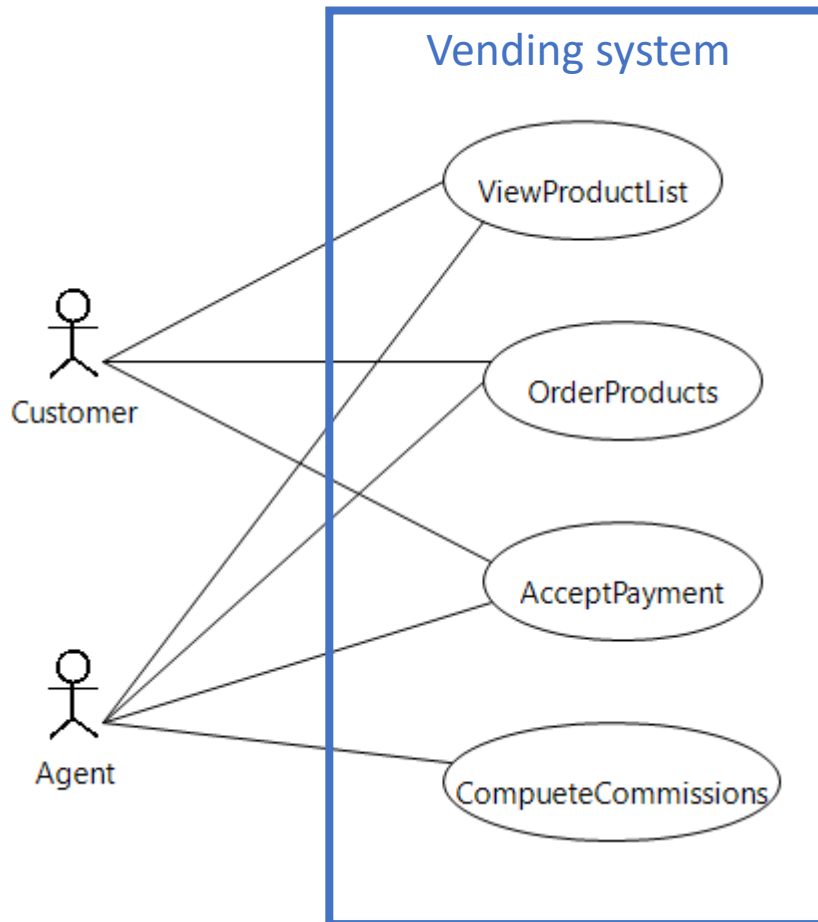


Example



System
composed of 4
use cases

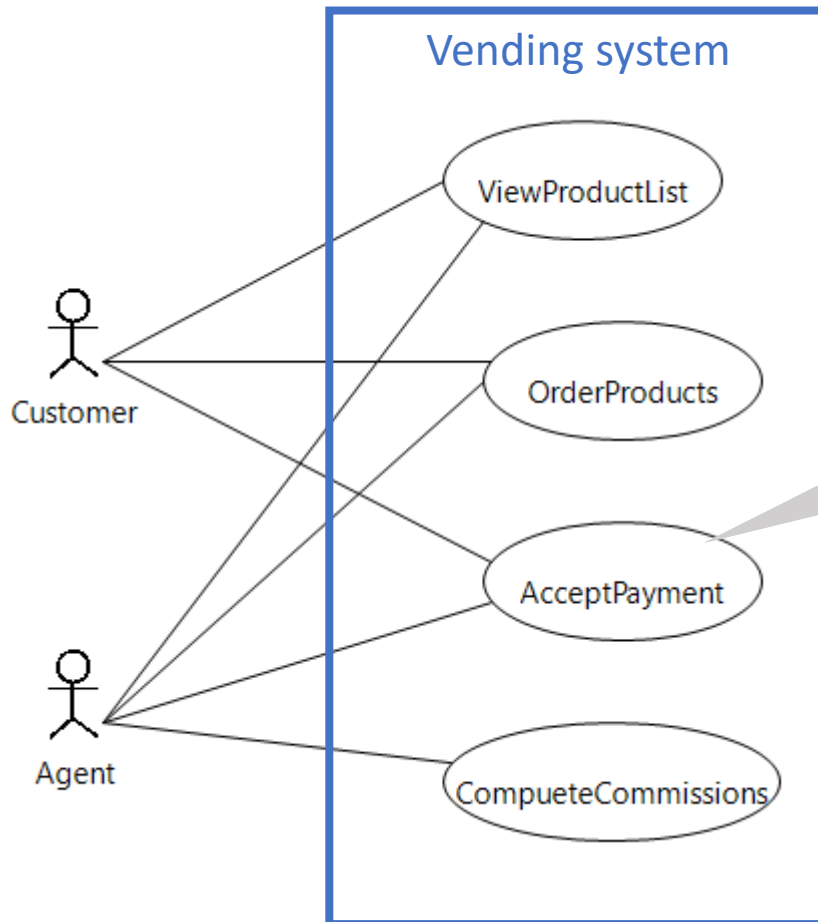
Example



System
composed of 4
use cases

It contains an
imprecision ...
which one?

Example



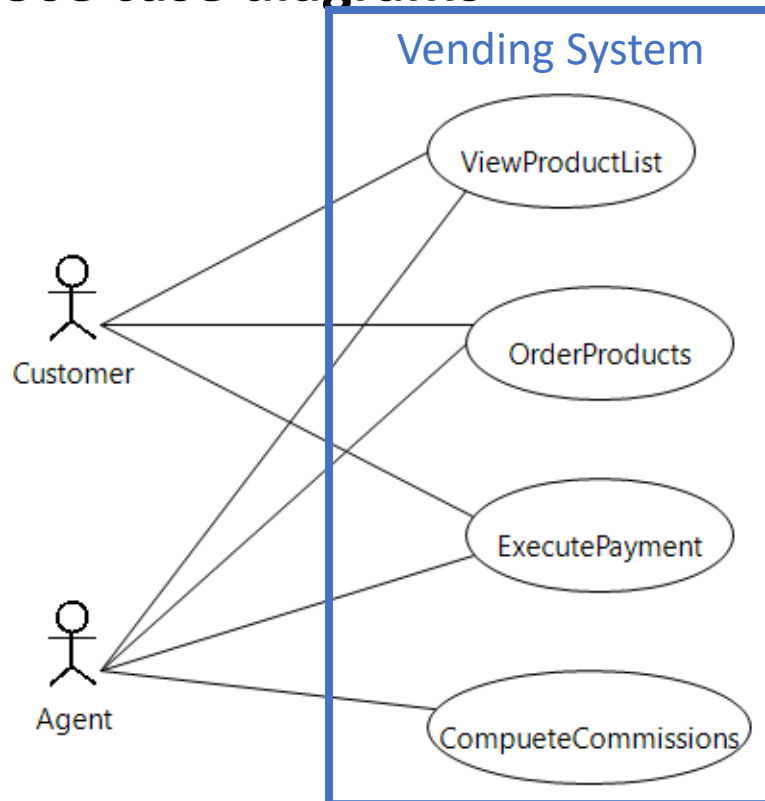
System
composed of 4
use cases

ExecutePayment

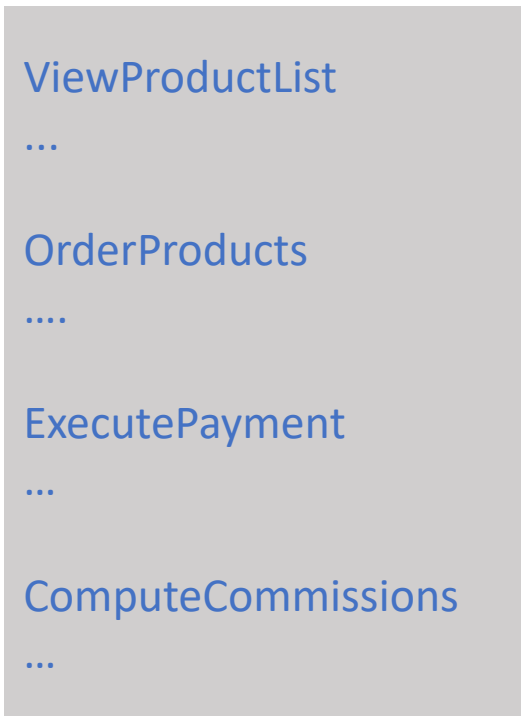
It contains an
imprecision ...
which one?

And then?

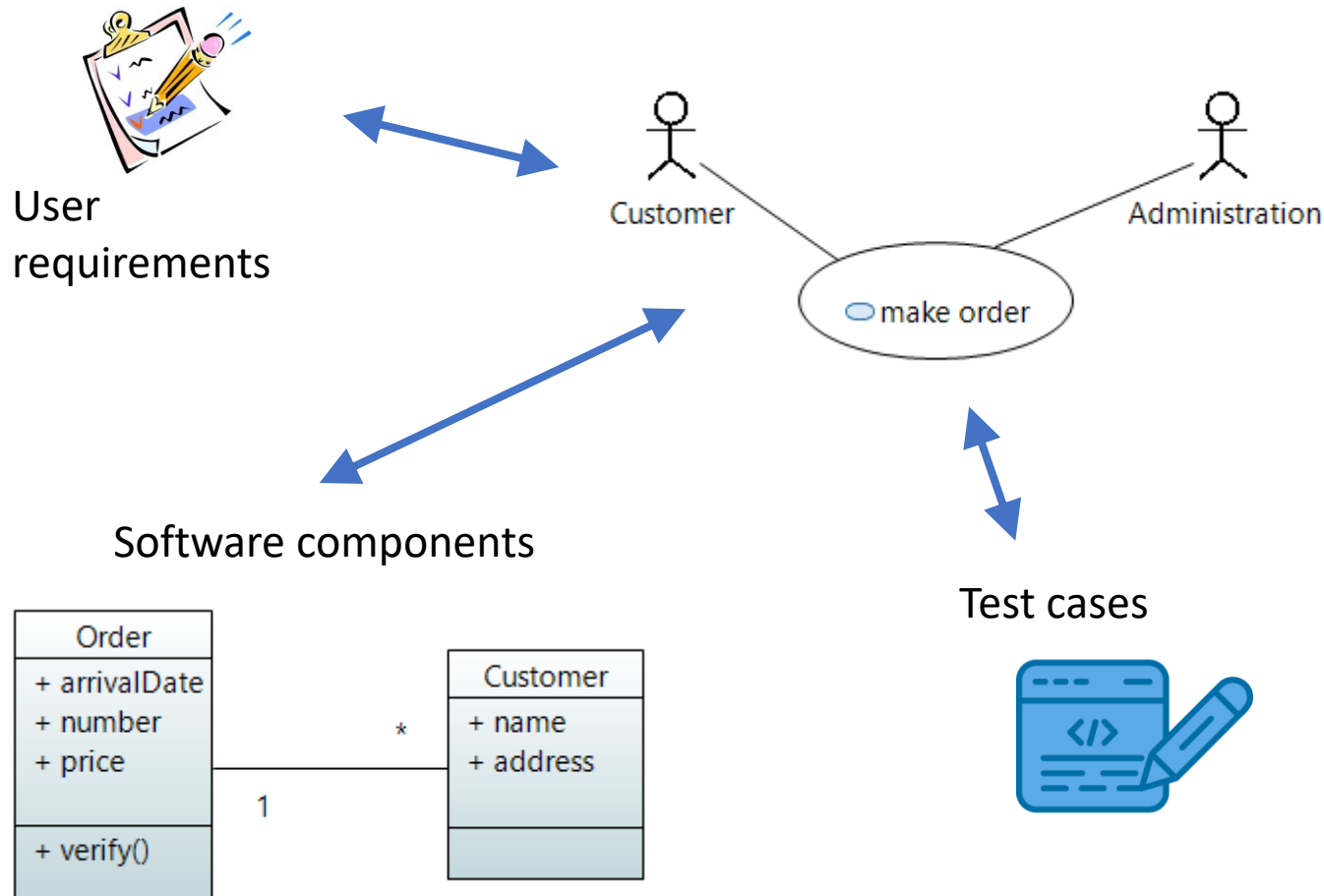
Use case diagrams



Use case descriptions



Useful for ...?



Benefits of use case diagrams

- **Easy to understand** for users -> useful to communicate and obtain the validation of the requirements
- Allow to clearly define the **boundaries** of the system and make it clear the different actors involved
- Describe the **behaviour of the system**, making clear issues in the requirement analysis phase
- Offer a very **good documentation**

Example use case: wash hands

- How a beginner would write the use case



Example use case: wash hands

- How an experienced developer writes a use case

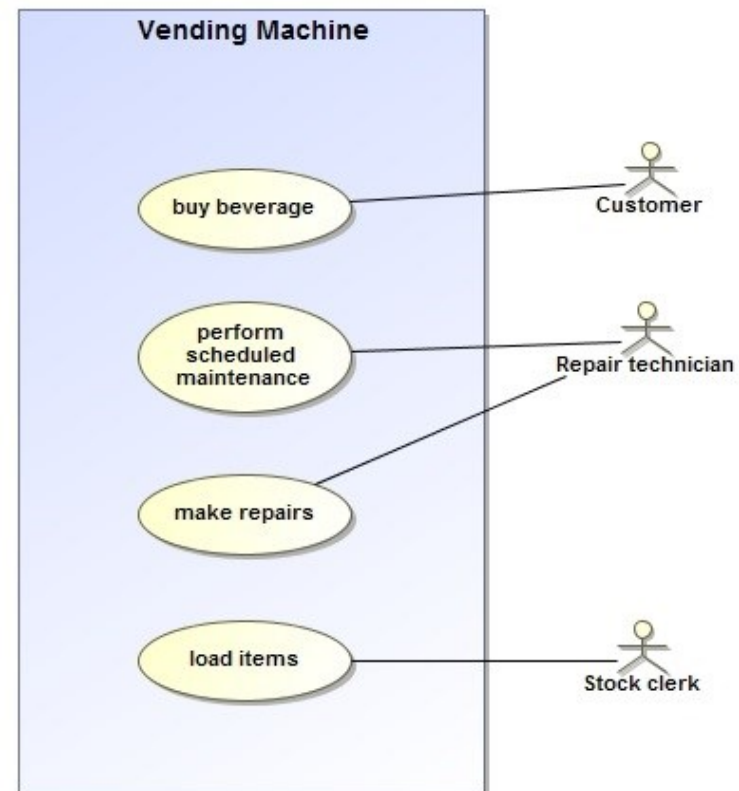


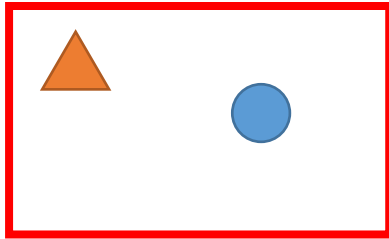
A good use case

- Starts with a request from an actor to the system
- Defines the interactions between system and actors
- Ends with an answer to the requests
- Takes the actor's point of view and not the system's one

Use case diagram: notation recap

- A rectangle (**system boundaries**) contains the use cases for a system with the actors listed on the outside. The name of the system is written near a side of the rectangle
- A “stick man” icon denotes an **actor** with the name placed below the icon
- A name within an ellipse denotes a **use case**
- Solid lines connect use cases to participating actors (**actor-case associations**)





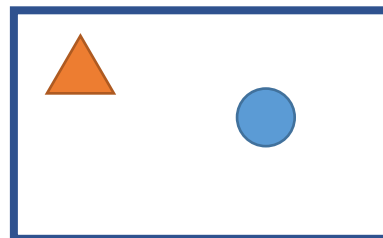
Use case
boundaries

Boundaries of a system

- The first step to carry out is **identifying the boundaries** of the system
 - Identify what is part of the system (inside the boundaries) and what is not (outside the boundaries)
- The correct identification of the boundaries is of the utmost importance for the functional requirements
- Several problems raise because of boundaries non-well defined
 - Clarify what is external (maybe other existing systems) and what should be carried out by the system that we are developing!



What already exists
and it does not have
to be developed



Boundaries of the
system



Actors

Actor

- A direct **external user** of a system
 - An object or a set of objects that communicates directly with the system but that is not part of the system
- Customer and Repair Technician are actors of a vending machine
 - Traveler, Agent and Airline are actors of a travel agency system
 - User and Administrator are actors for a computer database system



Actors

- Can be persons, devices and other systems (anything that interacts directly with the system)



Application under development

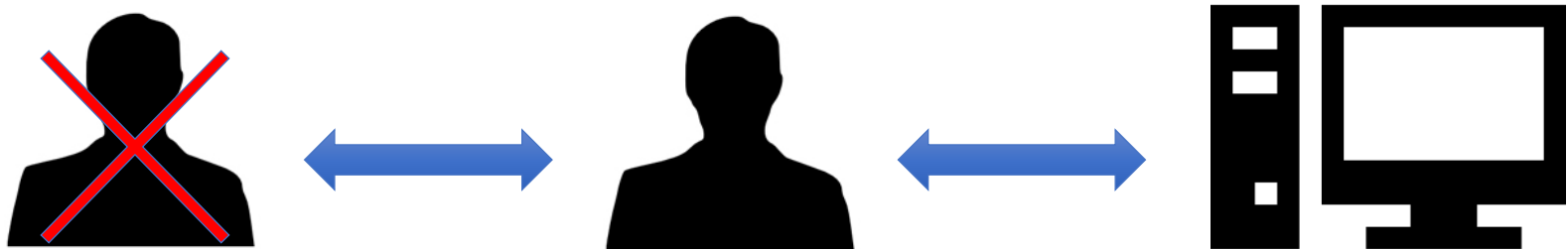
- In an online shop the use case “Checkout” involves the Web Customer and the Credit Payment Service

Actors

- An actor represents a particular facet (i.e., **role**) of objects in its interaction with a system
- A role is like a hat that a person can wear in a particular context
 - Many individual persons can be a User actor (e.g., Alice, Bob, ...) but they could also be other actors
- The same actor can represent different objects that interact similarly with a system
 - Many individual persons may use a vending machine but their behavior toward the vending machine can be summarized by the actors Customer and Repair Technician
 - Each actor represents a coherent set of capabilities for its objects

Actors

- Modelling the actors helps to define a system by identifying the objects within the system and those on its boundary
- An actor is **directly connected to the system**. An indirectly connected object is not an actor and should not be included as part of the system model
 - Example: the Dispatcher of repair technicians from a service bureau is not an actor of a vending machine



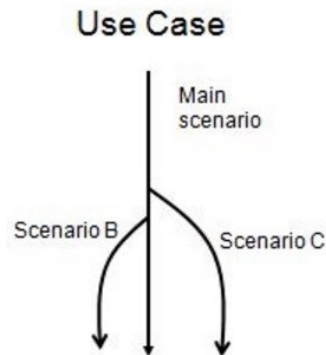
Actors

- Can be primary or secondary actors
 - **Primary**: use the proper functionalities of the system (active actors)
 - **Secondary**: leverage information or notifications produced by the use case (passive actors)
- An actor can be active in a use case and passive in another one

Finding actors

- The actors of a system can be identified by answering a number of questions:
 - Who will use the functionality of the system?
 - Who will maintain the system?
 - What devices does the system need to handle?
 - What other system does this system need to interact?
 - Who or what has interest in the results of this system?





Use cases and scenarios

Use case

- A use case is a coherent **piece of functionality** that a system can provide by interacting with actors

- **Buy a beverage.** The vending machine delivers a beverage after a customer selects and pays for it.
- **Perform scheduled maintenance.** A repair technician performs the periodic service on the vending machine necessary to keep it in good working condition.
- **Make repairs.** A repair technician performs the unexpected service on the vending machine necessary to repair a problem in its operation.
- **Load items.** A stock clerk adds items into the vending machine to replenish its stock of beverages.

Figure 7.1 Use case summaries for a vending machine. A use case is a coherent piece of functionality that a system can provide by interacting with actors.

Object-Oriented Modeling and Design with UML, Second Edition by Michael Blaha and James Rumbaugh. ISBN 0-13-1-015920-4. © 2005 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Use cases

- Each use case involves one or more actors as well as the system itself
 - The use case “Buy a beverage” involves the Customer
 - In a telephone system the use case “Make a call” involves two actors, a Caller and a Receiver
- Use cases should all be at a comparable level of abstraction
 - “Make telephone call” and “Record voice mail message” are at a comparable level
 - “Set external speaker volume to high” is too narrow, “Set speaker volume” or even “Set telephone parameters” would be better

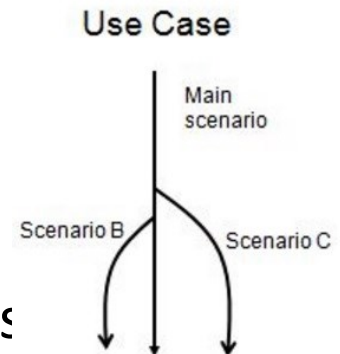
Finding use cases

- For each actor ask these questions:
 - Which functions does the actor require from the system?
 - What does the actor need to do?
 - Could the actor's work be simplified or made efficient by new functions in the system?
 - What are the problems with the existing systems?
 - What are the inputs and outputs of the system?



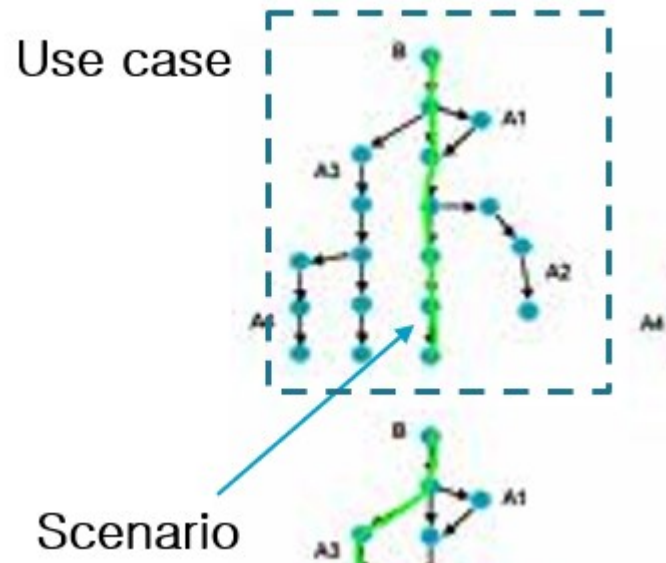
Use case description

- It describes a flow of events (scenario) modelling the behaviour of the use case
 - **Main flow** (basic flow or happy path): what usually happens (most of the times)
 - The perfect world, where everything happens as desired
 - **Alternative flows**: exceptional or error situations
 - Errors
 - Special cases
 - Other possible actions or behaviours

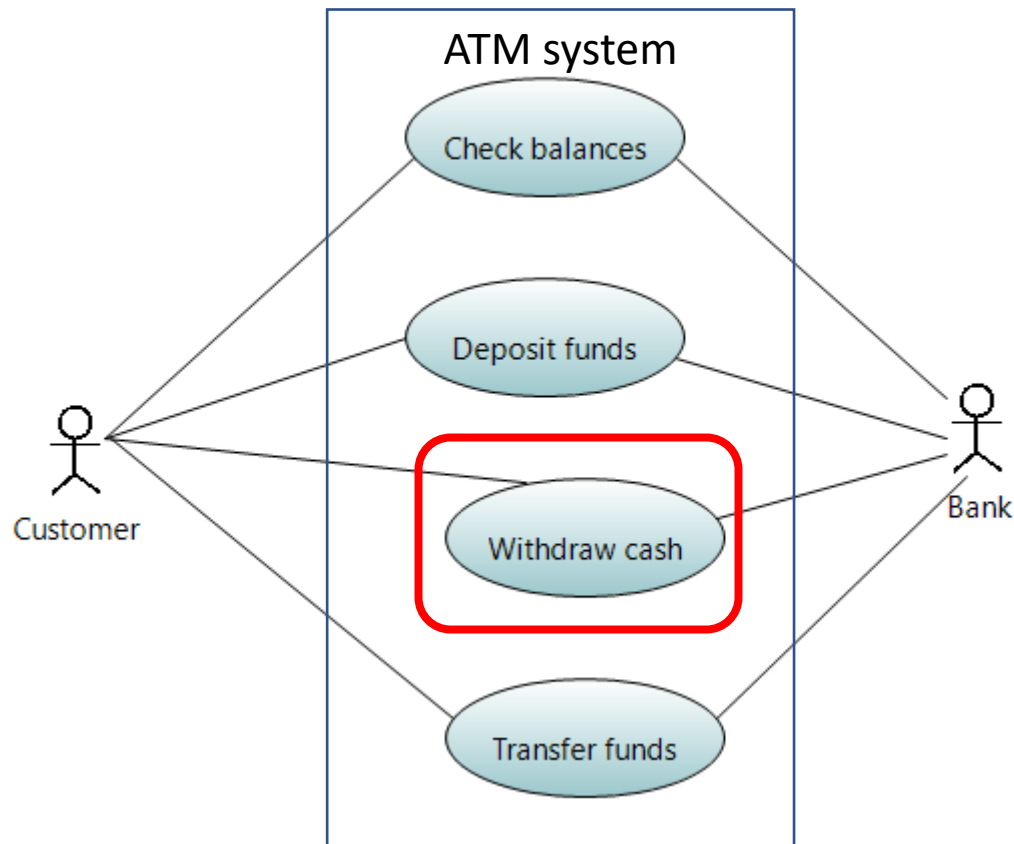


Use cases and scenarios

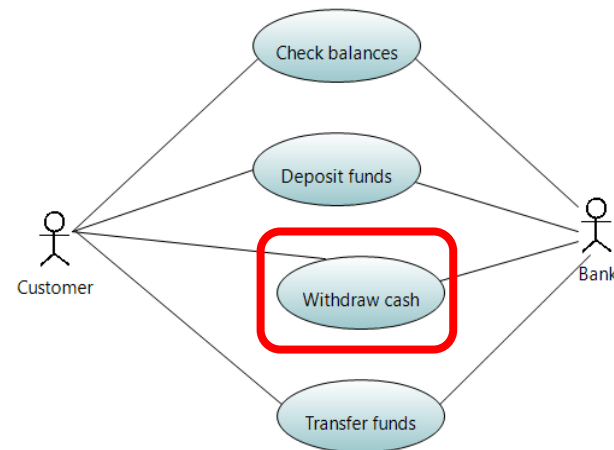
- A scenario represents a single path through the use case



Example: cash withdrawal



Scenario: withdraw cash



- An ordered sequence of interactions between system and actors
- A specific execution of a use case (a path of the use case)
- However, this is not the only possible scenario
- Different scenarios, but in all cases the user has the same objective: cash withdrawal (the use case)

1. The customer inserts the debit card and enters the PIN
2. Customer is validated
3. Customer enters desired amount
4. Information is sent to the Bank, inquiring of sufficient funds/allowable withdrawal limit
5. Money is dispensed and receipt printed

- Incorrect bank account or PIN
- Requested amount not available on the account or in the machine
- The customer does not confirm

Use case description

- The event flow can be described in a semi-structured textual format or through activities (sequence/activity/BPMN diagrams)
- Usually through structured text (form/template)

Use Case 1.1 *Withdraw money from an ATM*

Withdraw money from an ATM

Primary actors: Customer
ATM Technician
Bank

Preconditions: Network connection is active
ATM has available cash

Basic flow of events:

1. Bank customer inserts debit card and enters PIN.
2. Customer is validated.
3. ATM displays actions available on ATM unit. Customer selects Withdraw Cash.
4. ATM prompts account.
5. Customer selects account.
6. ATM prompts amount.
7. Customer enters desired amount.
8. Information sent to Bank, inquiring if sufficient funds/allowable withdrawal limit.
9. Money is dispensed and receipt prints.

Alternative flows

- 2a. Customer is not validated.
 - 2a1. ATM displays error message.
- 7a. Customer selects invalid amount.
 - 7a1. ATM prompts user to re-enter valid amount.
- 8a. Customer has insufficient funds.
 - 8a1. ATM displays error message.
 - 8a2. ATM shows available withdrawal limit, redirects to step 6.
- 9a. ATM has insufficient cash.
 - 9a1. ATM Technician is alerted.
 - 9a2. ATM displays error message and phone number to call.
- 9b. Cash gets stuck in dispensing.
 - 9b1. ATM displays error message.

Use case description

- The steps are usually simple and easy to understand
 - They have to be understood by the customer
- Descriptions clear, precise, general and independent of the technological aspects

Use Case 1.1 *Withdraw money from an ATM*

Withdraw money from an ATM

Primary actors: Customer
ATM Technician
Bank

Preconditions: Network connection is active
ATM has available cash

Basic flow of events:

1. Bank customer inserts debit card and enters PIN.
2. Customer is validated.
3. ATM displays actions available on ATM unit. Customer selects Withdraw Cash.
4. ATM prompts account.
5. Customer selects account.
6. ATM prompts amount.
7. Customer enters desired amount.
8. Information sent to Bank, inquiring if sufficient funds/allowable withdrawal limit.
9. Money is dispensed and receipt prints.

Alternative flows

- 2a. Customer is not validated.
 - 2a1. ATM displays error message.
- 7a. Customer selects invalid amount.
 - 7a1. ATM prompts user to re-enter valid amount.
- 8a. Customer has insufficient funds.
 - 8a1. ATM displays error message.
 - 8a2. ATM shows available withdrawal limit, redirects to step 6.
- 9a. ATM has insufficient cash.
 - 9a1. ATM Technician is alerted.
 - 9a2. ATM displays error message and phone number to call.
- 9b. Cash gets stuck in dispensing.
 - 9b1. ATM displays error message.

How do we describe a single step?

- Step

- <number> <subject (system/actor)> <Action>

Use Case 1.1 *Withdraw money from an ATM*

Withdraw money from an ATM

Primary actors: Customer

Preconditions: Network connection is active
ATM has available cash

Basic flow of events:

1. Bank customer inserts debit card and enters PIN.
2. Customer is validated.
3. ATM displays actions available on ATM unit. Customer selects Withdraw Cash.
4. ATM prompts account.
5. Customer selects account.
6. ATM prompts amount.
- ▶ 7. Customer enters desired amount.
- ~~8. Information sent to Bank, inquiring if sufficient funds/allowable withdrawal limit.~~
9. Money is dispensed and receipt prints.

Use case description

- Usually include
 - **Main flow**: perfect world (happy path) scenario
 - **Alternative flows**: exceptional situations in terms of
 - Errors
 - Special cases
 - Other types of behavior

Use Case 1.1 *Withdraw money from an ATM*

Withdraw money from an ATM

Primary actors: Customer
ATM Technician
Bank

Preconditions: Network connection is active
ATM has available cash

Basic flow of events:

1. Bank customer inserts debit card and enters PIN.
2. Customer is validated.
3. ATM displays actions available on ATM unit. Customer selects Withdraw Cash.
4. ATM prompts account.
5. Customer selects account.
6. ATM prompts amount.
7. Customer enters desired amount.
8. Information sent to Bank, inquiring if sufficient funds/allowable withdrawal limit.
9. Money is dispensed and receipt prints.

Alternative flows

- 2a. Customer is not validated.
2a1. ATM displays error message.
- 7a. Customer selects invalid amount.
7a1. ATM prompts user to re-enter valid amount.
- 8a. Customer has insufficient funds.
8a1. ATM displays error message.
8a2. ATM shows available withdrawal limit, redirects to step 6.
- 9a. ATM has insufficient cash.
9a1. ATM Technician is alerted.
9a2. ATM displays error message and phone number to call.
- 9b. Cash gets stuck in dispensing.
9b1. ATM displays error message.

Use case templates

- Several use case templates exist
 - Alistair Cockburn's template
 - Other versions

Alistair Cockburn use case template

USE CASE #	< the name is the goal as a short active verb phrase>	
Goal in Context	<a longer statement of the goal in context if needed>	
Scope & Level	<what system is being considered black box under design> <one of : Summary, Primary Task, Subfunction>	
Preconditions	<what we expect is already the state of the world>	
Success End Condition	<the state of the world upon successful completion>	
Failed End Condition	<the state of the world if goal abandoned>	
Primary, Secondary Actors	<a role name or description for the primary actor>. <other systems relied upon to accomplish use case>	
Trigger	<the action upon the system that starts the use case>	
DESCRIPTION	Step	Action
	1	<put here the steps of the scenario from trigger to goal delivery, and any cleanup after>
	2	<...>
	3	
EXTENSIONS	Step	Branching Action
	1a	<condition causing branching> : <action or name of sub.use case>
SUB-VARIATIONS		Branching Action
	1	<list of variation s>

Alistair Cockburn use case example

USE CASE 5	Buy Goods	
Goal in Context	Buyer issues request directly to our company, expects goods shipped and to be billed.	
Scope & Level	Company, Summary	
Preconditions	We know Buyer, their address, etc.	
Success End Condition	Buyer has goods, we have money for the goods.	
Failed End Condition	We have not sent the goods, Buyer has not spent the money.	
Primary, Secondary Actors	Buyer, any agent (or computer) acting for the customer. Credit card company, bank, shipping service	
Trigger	purchase request comes in.	
DESCRIPTION	Step	Action
	1	Buyer calls in with a purchase request
	2	Company captures buyer's name, address, requested goods, etc.
	3	Company gives buyer information on goods, prices, delivery dates, etc.
	4	Buyer signs for order.
	5	Company creates order, ships order to buyer.
	6	Company ships invoice to buyer.
	7	Buyers pays invoice.
EXTENSIONS	Step	Branching Action
	3a	Company is out of one of the ordered items: 3a1. Renegotiate order.
	4a	Buyer pays directly with credit card: 4a1. Take payment by credit card (use case 44)
	7a	Buyer returns goods: 7a. Handle returned goods (use case 105)
SUB-VARIATIONS		Branching Action
	1	Buyer may use phone in, fax in, use web order form, electronic interchange
	7	Buyer may pay by cash or money order check credit card

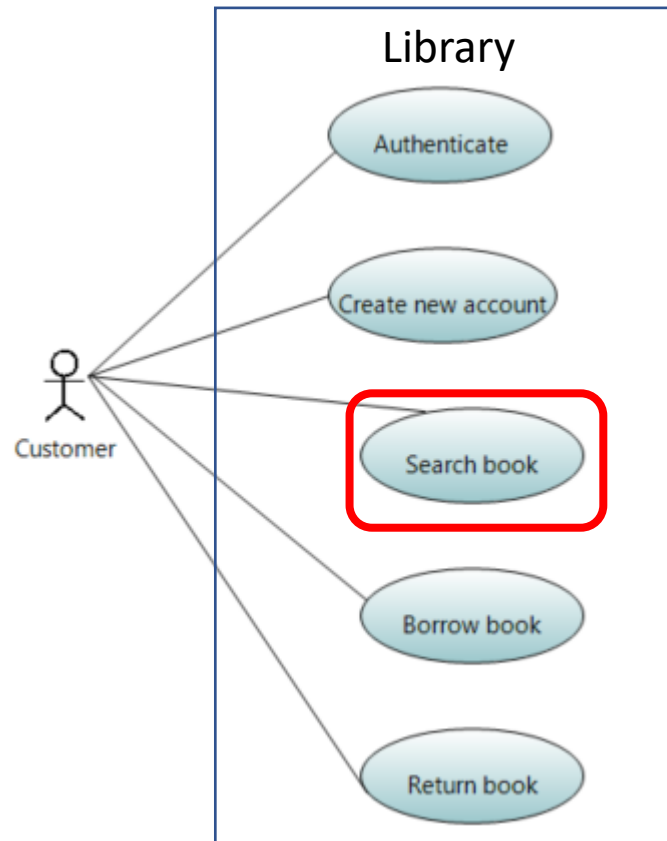
Typical use case template

- **Use case name:** use case goal (usually verb+object)
- **Use case Identifier:** typically progressive number
- **Short description:** typically a paragraph describing the objective of the use case
- **Primary actors**
- **Secondary actors**
- **Preconditions:** constraints on the current state of the system (they can be written in OCL)
- **Postconditions:** conditions that should be true once the use case completes successfully (they can be written in OCL)
- **Main scenario:** main flow
- **Alternative scenarios:** a list of alternatives to the main scenario

Use case template

Use case Name	
Id	
Description	
Primary actors	
Secondary actors	
Preconditions	
Main flow	
Postconditions	
Alternative flows	

Example: library

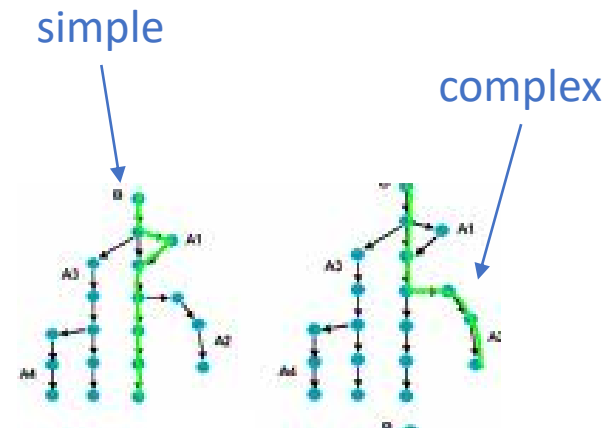


Search book in the library system

Use case Name	Search book
Id	UC5
Description	The customer searches for a book in the online library
Primary actors	Customer
Secondary actors	none
Preconditions	Customer is authenticated
Main flow	1. The customer selects the functionality “search book” 2. The system asks for the search criteria 3. The customer inserts the criteria and confirms 4. The system searches the books according the inserted criteria 5. The system shows the list of the books
Postconditions	none
Alternative flows	none

Deviations and repetitions

- We have a deviation every time we move away from the main scenario
- Deviations:
 - Simple: we use **if** and **otherwise** in the main sequence
 - Complex: we write alternative flows. They are usually errors or special cases that do not come back on the main flow
- Repetitions within a sequence
 - **Foreach** and **as long as**



Use case deviations

Use case Name	Search book_if
Id	UC5
Description	The customer searches for a book in the online library
Primary actors	Customer
Secondary actors	none
Preconditions	Customer is authenticated
Main flow	1. The customer selects the functionality “search book” 2. The system asks for the search criteria 3. The customer inserts the criteria and confirms 4. The system searches the books according the inserted criteria 5. If the system finds one or more books 5.1. The system shows the list of the books 6. Otherwise 6.1 The system communicates to the customer that there are no books satisfying the inserted requirements
Postconditions	none
Alternative flows	none

Use case repetitions (foreach)

Use case Name	Search book_ foreach
Id	UC5
Description	The customer searches for a book in the online library
Primary actors	Customer
Secondary actors	none
Preconditions	Customer is authenticated
Main flow	1. The customer selects the functionality “search book” 2. The system asks for the search criteria 3. The customer inserts the criteria and confirms 4. The system searches the books according the inserted criteria 5. If the system finds one or more books 5.1. Foreach book found 5.1.1 The system shows the picture of the book 5.1.2 The system shows the characteristics of the book 5.1.3 The system shows the price of the book 6. Otherwise 6.1 The system communicates to the customer that there are no books satisfying the inserted requirements
Postconditions	none
Alternative flows	none

Use case repetitions (as long as)

Use case Name	Search book as long as
Id	UC5
Description	The customer searches for a book in the online library
Primary actors	Customer
Secondary actors	none
Preconditions	Customer is authenticated
Main flow	1. The customer selects the functionality “search book” 2. The system asks for the search criteria 3. The customer inserts the criteria and confirms 4. As long as the search criteria are invalid 4.1. The system asks for inserting criteria again 4.2. The customer inserts the search criteria 4.3. The system checks the inserted criteria 5. The system searches the books according the inserted criteria 6. If the system finds one or more books 5.1. The system shoes the list of the books 7. Otherwise 6.1 The system communicates to the customer that there are no books satisfying the inserted requirements
Postconditions	none

Alternative scenarios

- Alternative scenarios can also be documented separately

Use case Name	Create New Account
Id	UC9
Description	The system creates a new account for the customer
Primary actors	Customer
Secondary actors	none
Preconditions	none
Main flow	1. The customer selects “create new account” 2. The system asks for the information of the customer, including email, password and password confirmation 3. The customer inserts the information and confirms 4. The system checks the inserted information 5. The system create a new account for the customer
Postconditions	A new account is created for the customer
Alternative flows	Invalid email address Invalid password Cancel

Alternative scenarios

Related to use case 9

Alternative flow	Create New Account: Invalid email address
Id	UC 9.1
Description	The system alerts the customer that he has inserted an invalid email address
Primary actors	Customer
Secondary actors	none
Preconditions	The customer has inserted an invalid email address
Main flow	The sequence of the steps starts after step 4 of the main flow - The system alerts the customer that has inserted an invalid email address
Postconditions	None

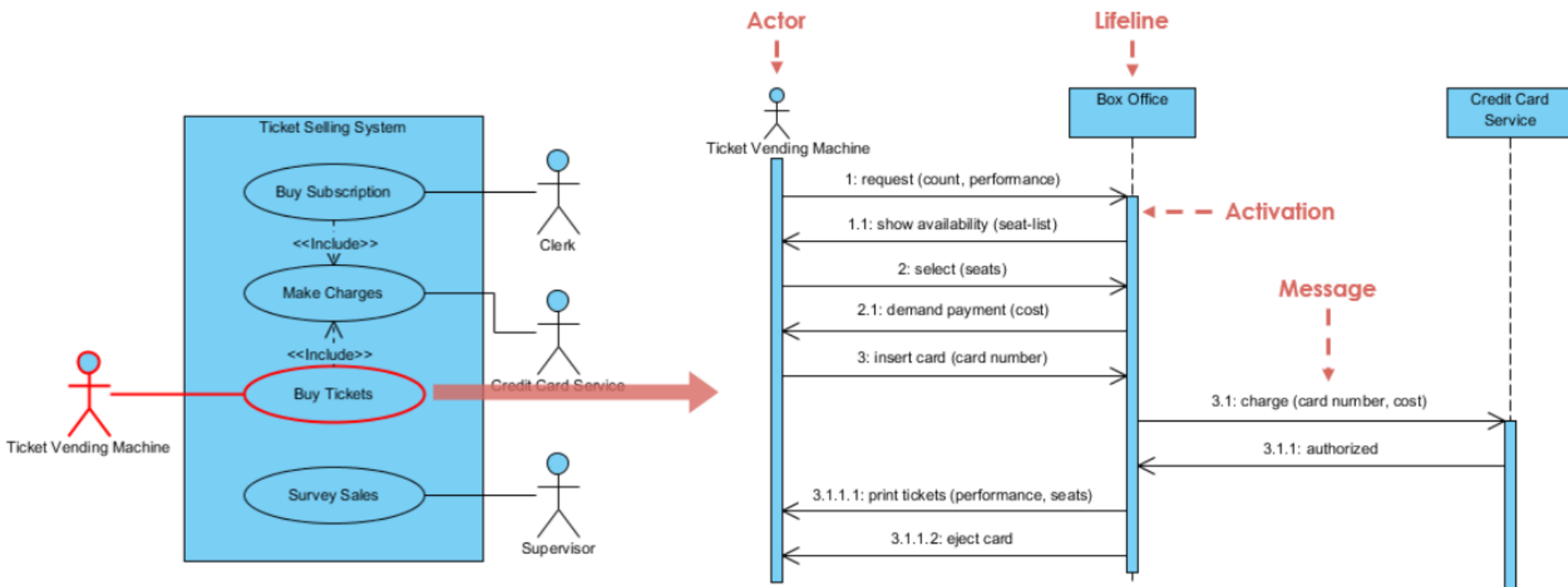
Specify the relationship with the main use case

The sequence of events can even be activated at anytime of the main sequence

Alternative scenarios

Alternative flow	Create New Account: Cancel
Id	UC 9.2
Description	The customer cancels the account creation process
Primary actors	Customer
Secondary actors	none
Preconditions	none
Main flow	1. The sequence of the steps starts at anytime in which the user has to perform an action 2. The customer cancels the account creation 3. The system shows the main page
Postconditions	None

Use case scenarios described with sequence diagrams



[<https://circle.visual-paradigm.com/use-case-system-level-sequence-diagram/>]

Use case scenarios described with activity diagrams

Use case name Create a new Blog Account

Related Requirements

Requirement A.1.

Goal In Context

A new or existing author requests a new blog account from the Administrator.

Preconditions

The system is limited to recognized authors, and so the author needs to have appropriate proof of identity.

Successful End Condition

A new blog account is created for the author.

Failed End Condition

The application for a new blog account is rejected.

Primary Actors

Administrator.

Secondary Actors

Author Credentials Database.

Trigger

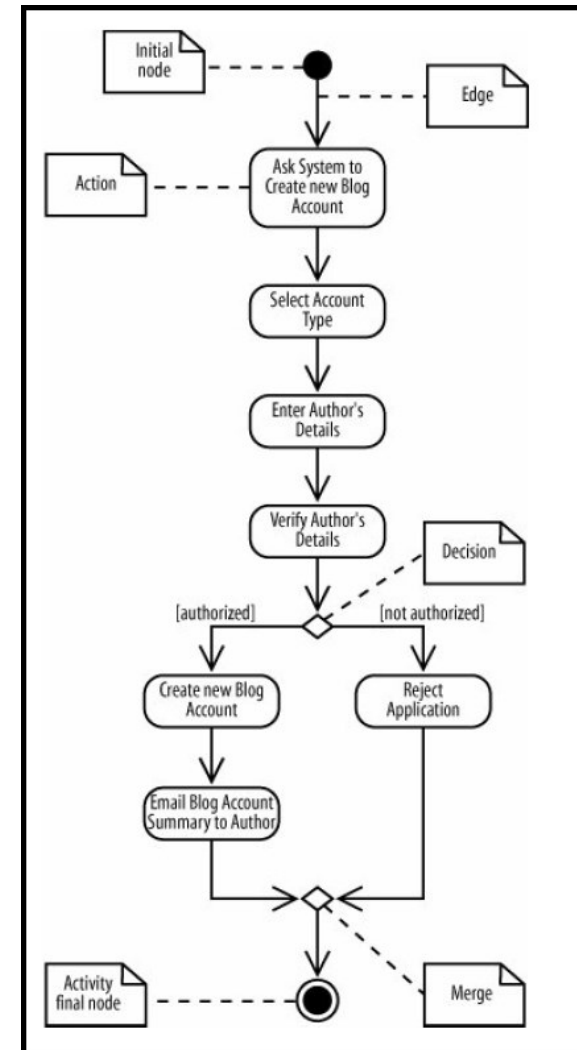
The Administrator asks the Content Management System to create a new blog account.

Main Flow

- | Step | Action |
|------|--|
| 1 | The Administrator asks the system to create a new blog account. |
| 2 | The Administrator selects an account type. |
| 3 | The Administrator enters the author's details. |
| 4 | The author's details are verified using the Author Credentials Database. |
| 5 | The new blog account is created. |
| 6 | A summary of the new blog account's details are emailed to the author. |

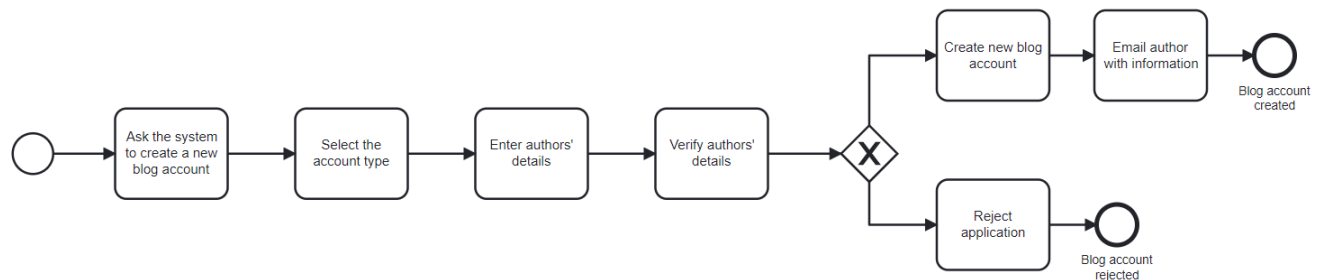
Extensions

- | Step | Branching Action |
|------|---|
| 4.1 | The Author Credentials Database does not verify the author's details. |
| 4.2 | The author's new blog account application is rejected. |



Use case scenarios described with BPMN models – low level detail

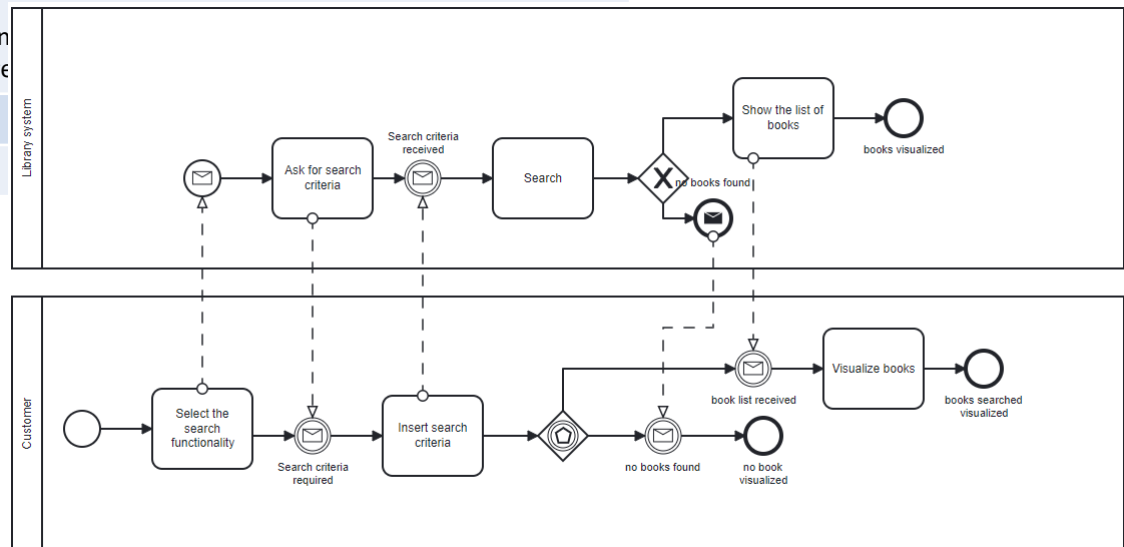
Use case name	Create a new Blog Account														
Related Requirements	Requirement A.1.														
Goal In Context	A new or existing author requests a new blog account from the Administrator.														
Preconditions	The system is limited to recognized authors, and so the author needs to have appropriate proof of identity.														
Successful End Condition	A new blog account is created for the author.														
Failed End Condition	The application for a new blog account is rejected.														
Primary Actors	Administrator.														
Secondary Actors	Author Credentials Database.														
Trigger	The Administrator asks the Content Management System to create a new blog account.														
Main Flow	<table border="1"> <thead> <tr> <th>Step</th> <th>Action</th> </tr> </thead> <tbody> <tr> <td>1</td> <td>The Administrator asks the system to create a new blog account.</td> </tr> <tr> <td>2</td> <td>The Administrator selects an account type.</td> </tr> <tr> <td>3</td> <td>The Administrator enters the author's details.</td> </tr> <tr> <td>4</td> <td>The author's details are verified using the Author Credentials Database.</td> </tr> <tr> <td>5</td> <td>The new blog account is created.</td> </tr> <tr> <td>6</td> <td>A summary of the author's details is sent to the author.</td> </tr> </tbody> </table>	Step	Action	1	The Administrator asks the system to create a new blog account.	2	The Administrator selects an account type.	3	The Administrator enters the author's details.	4	The author's details are verified using the Author Credentials Database.	5	The new blog account is created.	6	A summary of the author's details is sent to the author.
Step	Action														
1	The Administrator asks the system to create a new blog account.														
2	The Administrator selects an account type.														
3	The Administrator enters the author's details.														
4	The author's details are verified using the Author Credentials Database.														
5	The new blog account is created.														
6	A summary of the author's details is sent to the author.														
Extensions	<table border="1"> <thead> <tr> <th>Step</th> <th>Branching Action</th> </tr> </thead> <tbody> <tr> <td>4.1</td> <td>The Author Credentials Database returns a rejection.</td> </tr> <tr> <td>4.2</td> <td>The author's details are not verified.</td> </tr> </tbody> </table>	Step	Branching Action	4.1	The Author Credentials Database returns a rejection.	4.2	The author's details are not verified.								
Step	Branching Action														
4.1	The Author Credentials Database returns a rejection.														
4.2	The author's details are not verified.														



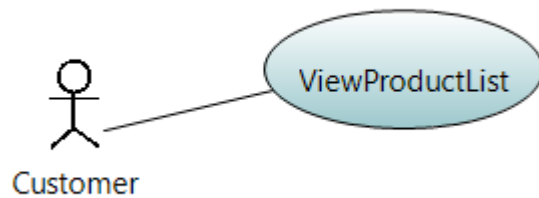
We will see BPMN for representing the dynamic behavior next week

Use case scenarios described with BPMN models – low level detail

Use case Name	Search book_if
Id	UC5
Description	The customer searches for a book in the online library
Primary actors	Customer
Secondary actors	none
Preconditions	Customer is authenticated
Main flow	1. The customer selects the functionality “search book” 2. The system asks for the search criteria 3. The customer inserts the criteria and confirms 4. The system searches the books according the inserted criteria 5. If the system finds one or more books 5.1. The system shows the list of the books 6. Otherwise 6.1 The system communicates satisfying the inserted re
Postconditions	none
Alternative flows	none



We will see BPMN for representing the dynamic behavior next week



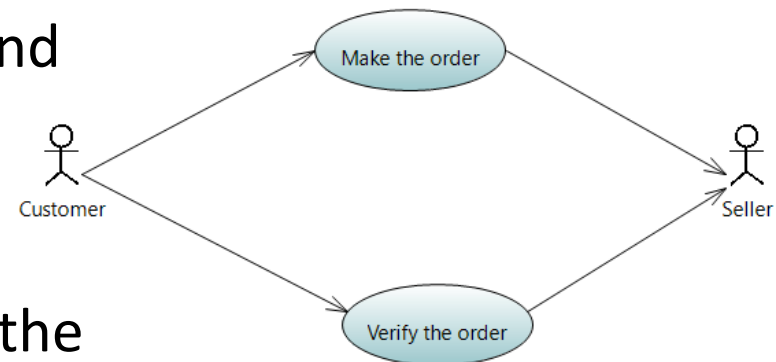
Actor – use case associations

Actors and use case

- Actors are connected to use cases through the **association relationship**
- The association relationship allows for activating (active actor) the use case or to receive (passive actor) the results

Association relationship

- Relation between the actor and the use case
- It can be a directed line (arrow) which indicates the **direction of the communication** between use case and actor
- For communications involving an answer it is not required adding an arrow towards the actor who starts the use case
- If the initial communication happens in both the directions both arrows are omitted

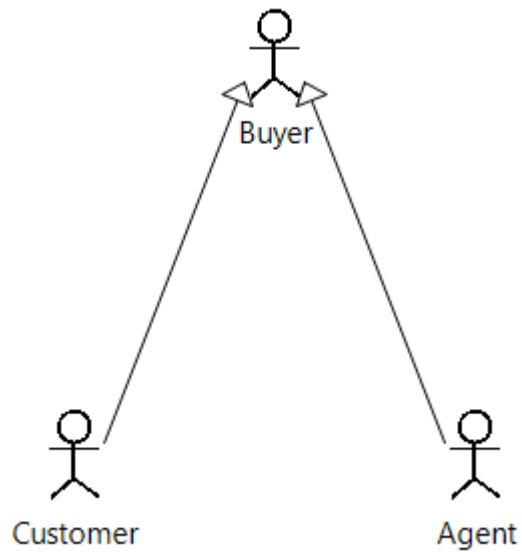




Exercise: reservation cancellation

- CarRen is an on-demand car rental service that allows you to request a private driver through a device application iPhone and Android.
- The service provides a cashless payment solution that charges the fare and tip or any other expense directly to the credit card associated with your account.
- *Draw the use case diagram (and the related use case description) related to the deletion functionality of a reservation that does not require any charge if the cancellation takes place within 5 minutes from the reservation and will be charged 10 euros otherwise.*

[inspired by http://didawiki.di.unipi.it/lib/exe/fetch.php/informatica/is-a/is_05_e1_esercitazionereq_casid_uso.pdf]



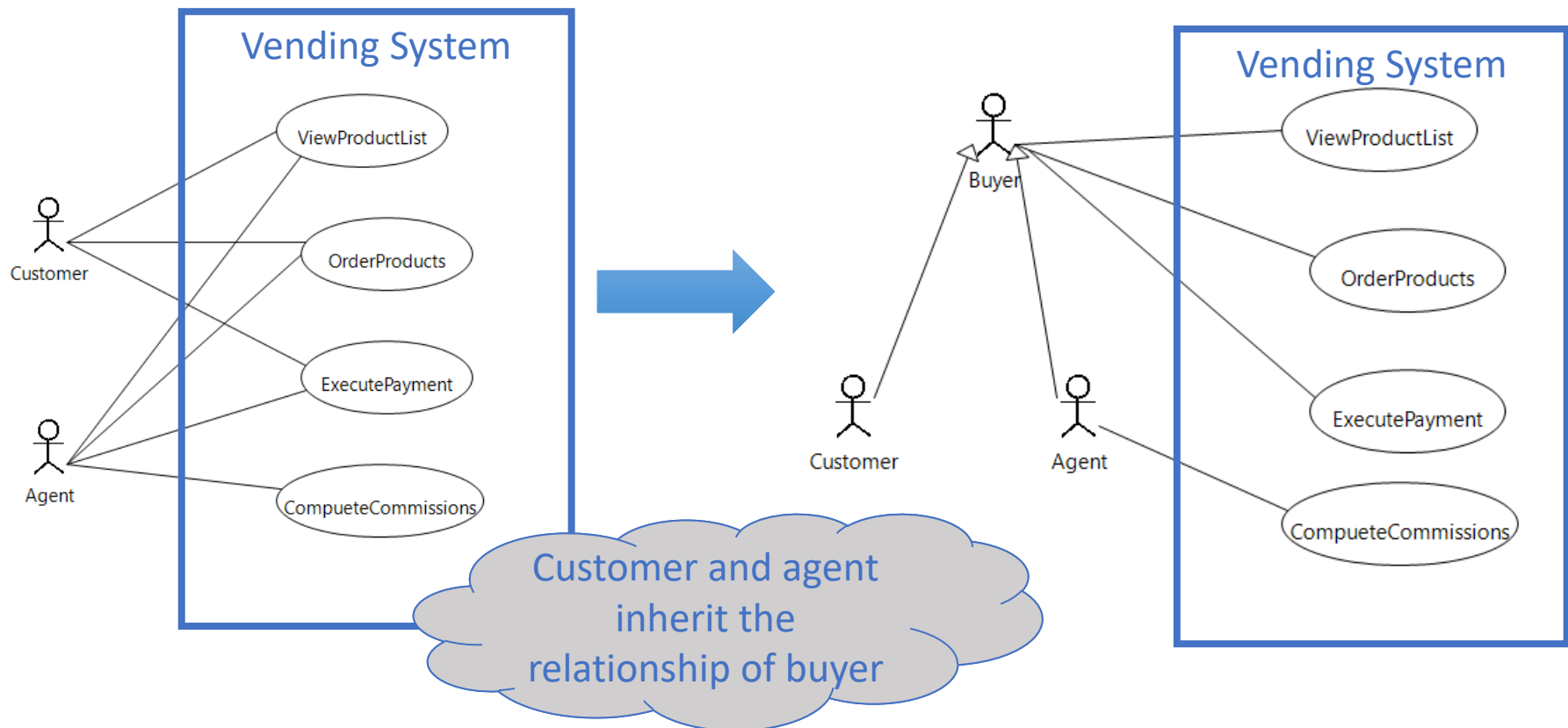
Actor generalizations

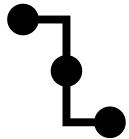
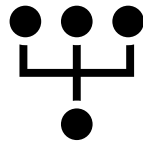
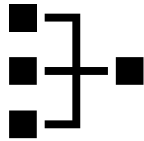
Actor generalization

- It can happen that different actors have similarities
- The generalization relationship can be used in order to create an **actor parent** (containing the similarities) and some children actors
- The **child** actor **inherits** all use case associations from the parent
- Actor generalization should be used if the specific actor has more responsibility than the generalized one (i.e., associated with more use cases)

Actor generalization

- The actor generalization is represented as an open arrow from the actor child to the actor parent





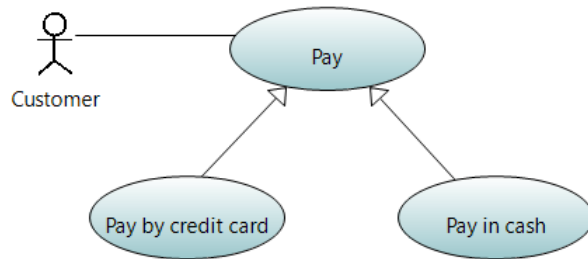
Use case relationships

Use cases

- Use cases are used only to describe functionalities (WHAT and NOT HOW)
- However
 - Functionalities can have different levels of abstraction
 - Some functionalities are similar
 - Some functionalities could require other functionalities as precondition
 - ...

Use case relationships

- For large applications complex use cases can be built from smaller pieces
- This enables flexibility in requirements specification
 - Isolating functionalities
 - Enabling functionality sharing
 - Breaking functionality into manageable chunks
- Three mechanisms are typically used:
 - Generalization
 - Inclusion
 - Extension



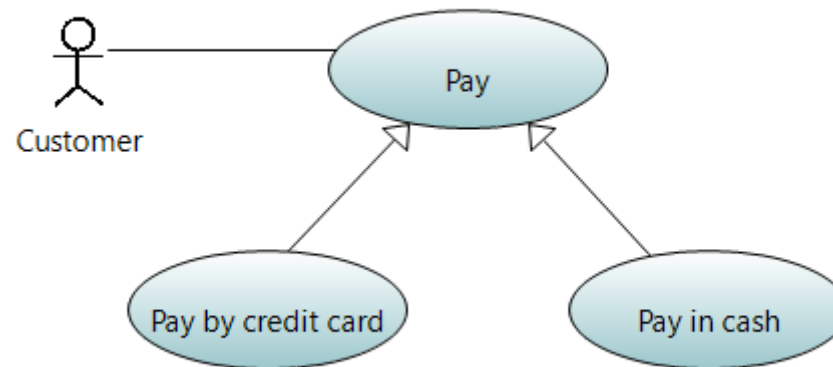
Generalization

Generalization relationship

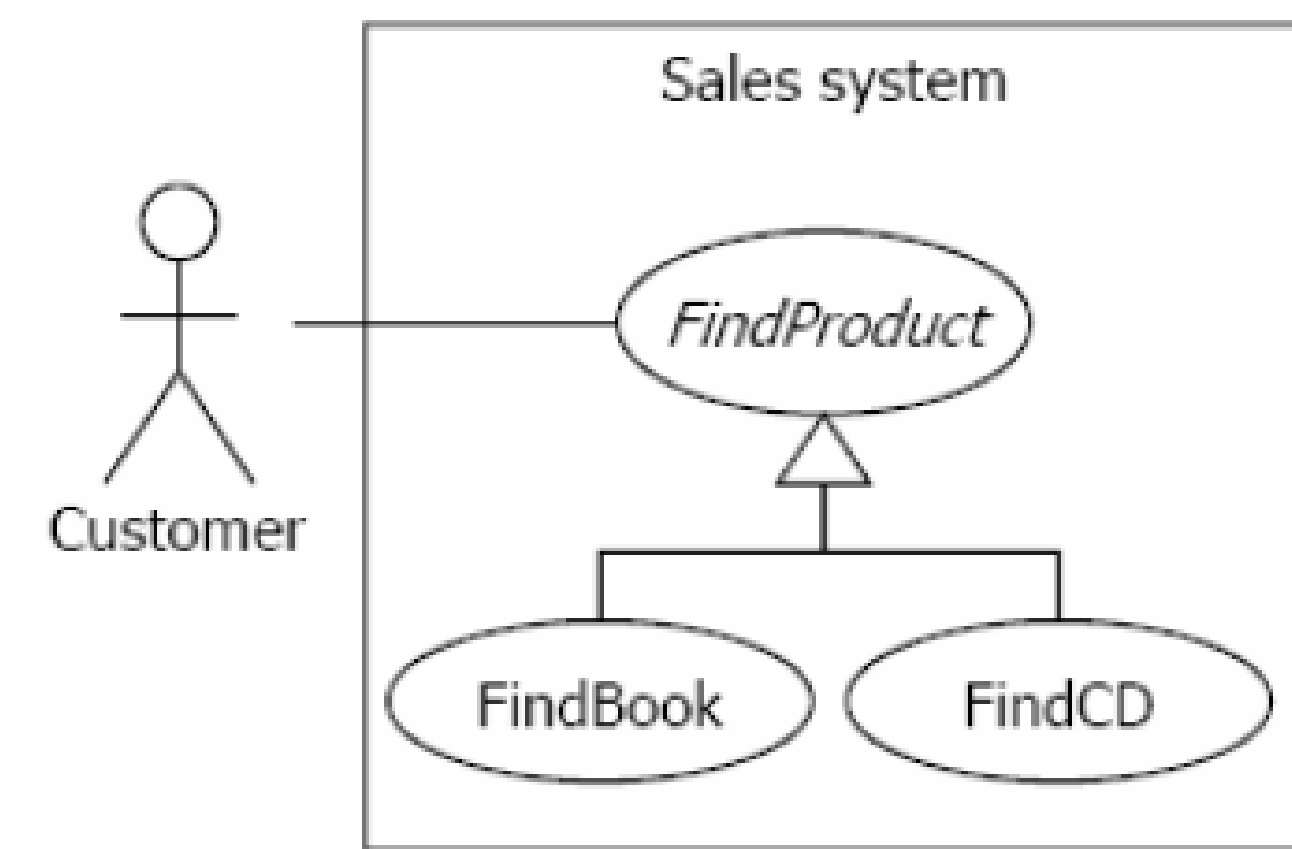
- It can happen that different use cases have similarities and share the same behaviour
- The generalization relationship can be used in order to create a use case parent (containing the similarities) and some children use cases
- The children
 - Inherit the steps of the parent
 - Can add new steps
 - Can redefine the inherited steps
 - Are mutually exclusive

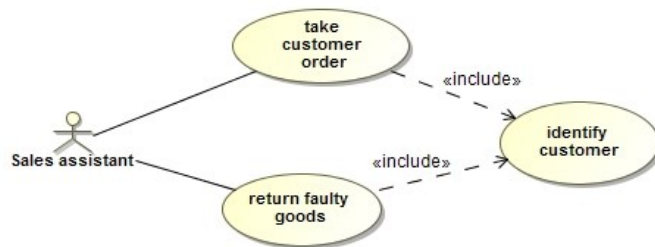
Generalization relationship notation

- The generalization notation is represented as an open arrow from the use case child to the use case parent



Generalization example





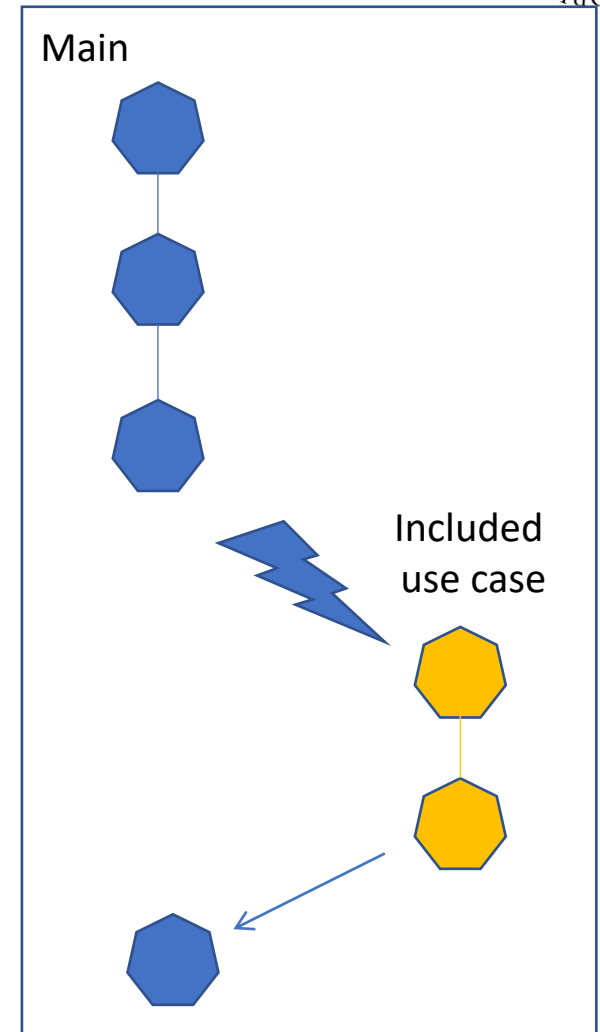
Include

Include relationship

- Include relationship
 - A use case can make use of other smaller use cases
 - The include relationship incorporates the behaviour of another use case (e.g., subroutines)
- Factoring a use case into pieces is appropriate when the pieces represent significant behaviour units

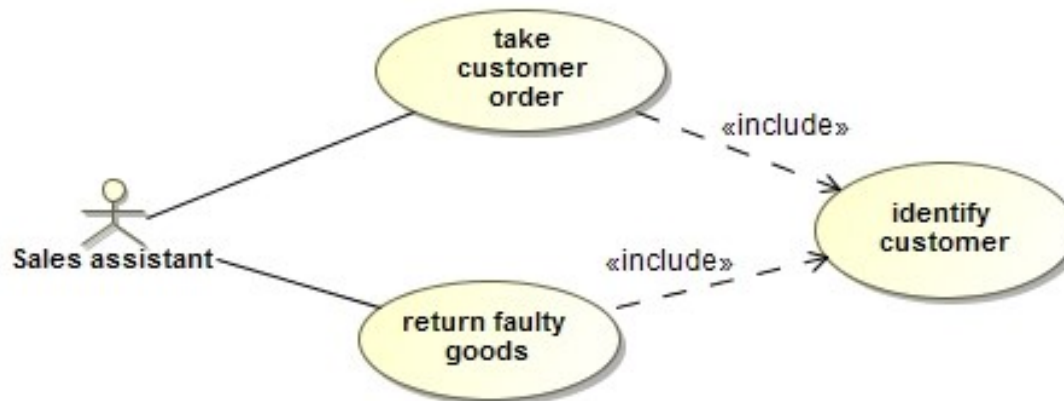
Include relationship

- It resembles the concept of procedure/function/macro...
- The “main” use case executes the steps up to the inclusion point and passes control to the included use case
- Eventually control returns to the main use case
- The main use case without an included use case is incomplete
- The included use case can be complete (i.e. instantiable) or be a fragment

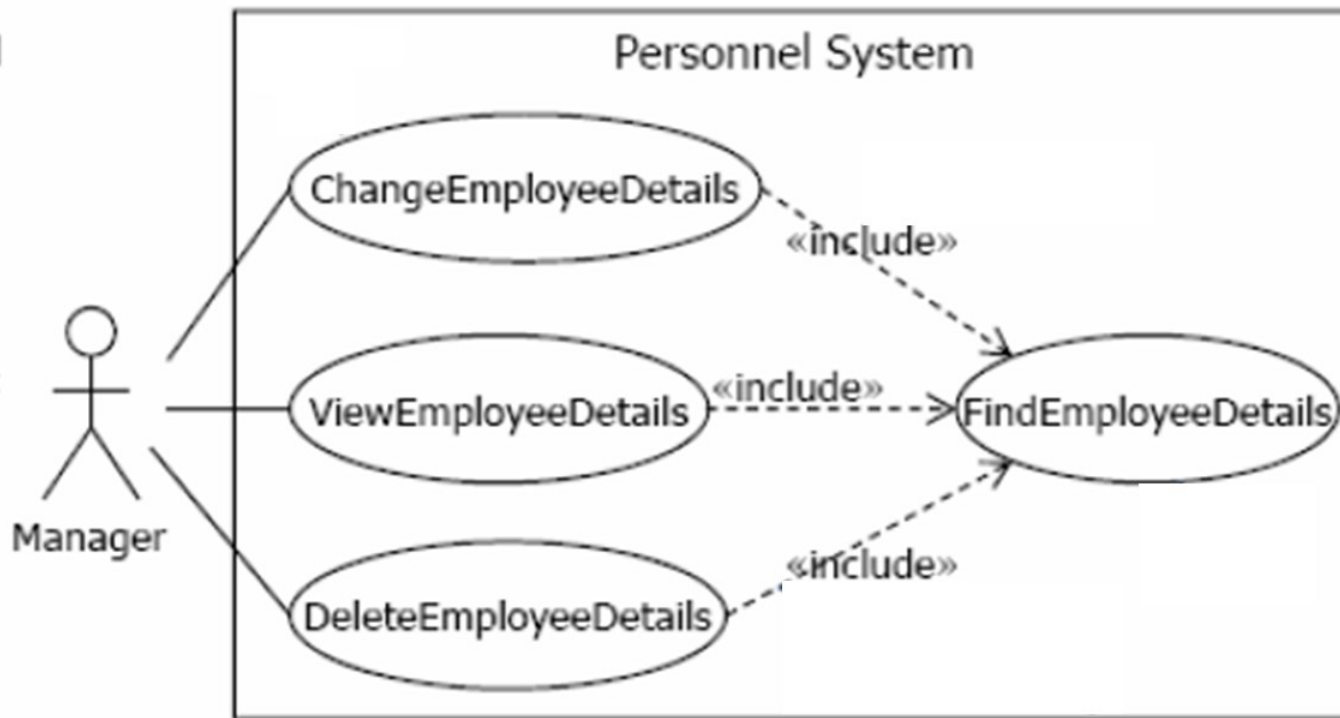


Include relationship notation

- Dashed arrow from the source (including) to the target (included) use case. The keyword «include» annotates the arrow



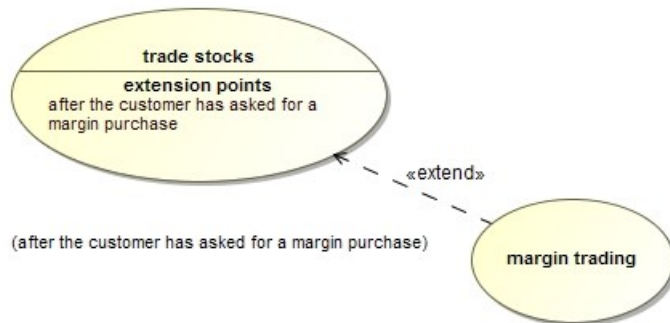
Include example



How can we represent it in the use case description?

Use case Name	Change Employee Details
Id	UC1
Description	The manager changes the employee details
Primary actors	Manager
Secondary actors	none
Preconditions	The manager is logged in the system
Main flow	1. Include(Find employee details) 2. The system displays the employee details 3. The manager changes the employee details 4. ...
Postconditions	none
Alternative flows	none

Use case Name	Find Employee Details
Id	UC4
Description	The manager finds the employee details
Primary actors	Manager
Secondary actors	none
Preconditions	The manager is logged in the system
Main flow	1. The manager enters the employee's ID 2. The system finds the employee details
Postconditions	The manager has found the employees details
Alternative flows	none



Extend

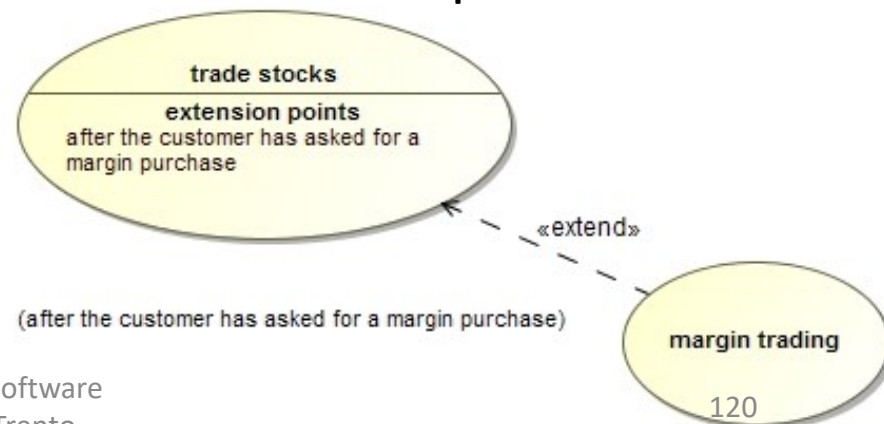
Extend

- Extend relationship
 - Adds an “extra behaviour” to/extends a base use case
 - used in the situation in which some initial capability is defined and later features are added modularly
- Base use case is meaningful and complete on its own, independently of the extension.
- Extension typically defines optional behavior that is not necessarily meaningful and complete by itself

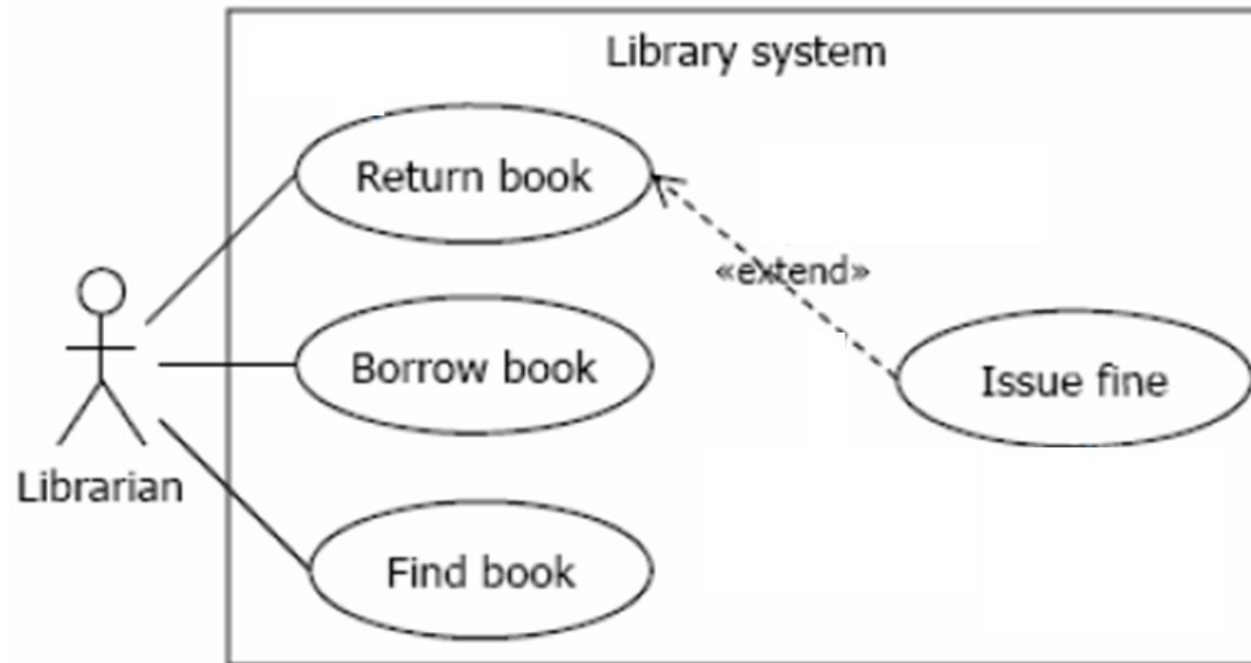
Extend relationship notation

- Dashed arrow from the extension to the base use case. The keyword <<extend>> annotates the arrow
- **Extension points**: specify the location at which the behavior of the base use case may be extended.
 - can have a condition attached.
 - The extension behaviour occurs only if the condition is true when the control reaches the extension point

Use case “trade stocks” is meaningful on its own. It could be optionally extended with “margin trading”

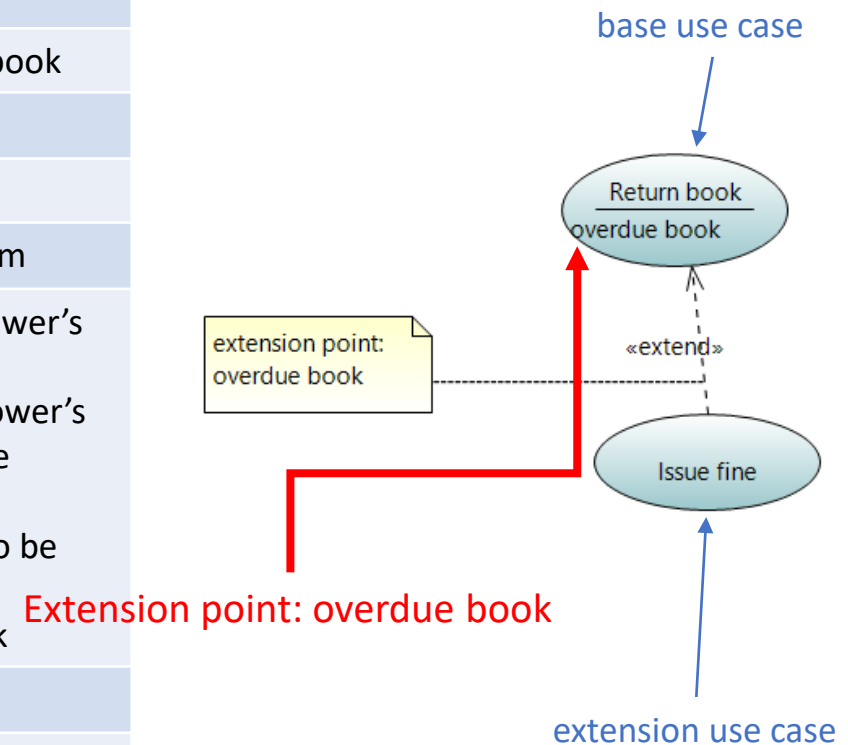


Extend example



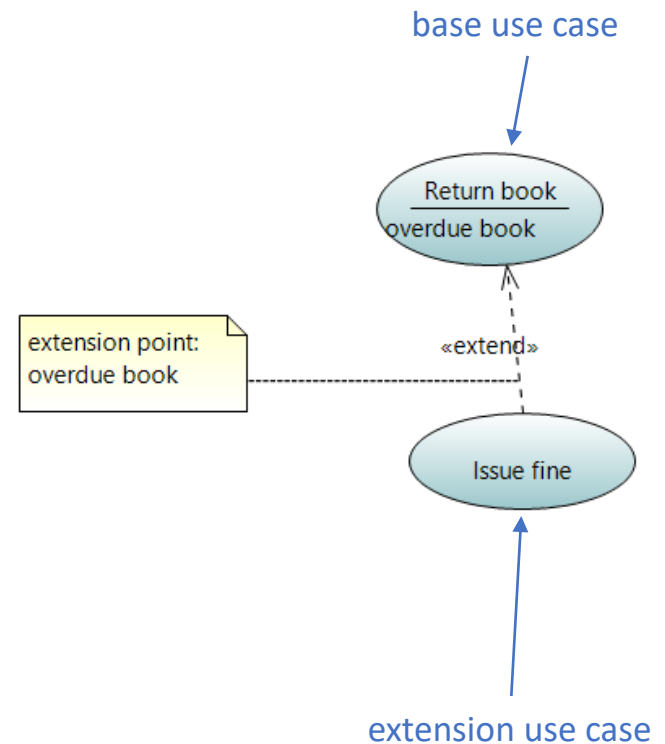
How do we represent it in the use case description?

Use case Name	Return Book
Id	UC9
Description	The librarian returns a borrowed book
Primary actors	Librarian
Secondary actors	none
Preconditions	The librarian is logged in the system
Main flow	1. The librarian enters the borrower's ID number 2. The system displays the borrower's details including the list of the borrowed books 3. The librarian finds the book to be returned in the list of books 4. The librarian returns the book
Postconditions	none
Alternative flows	none
Extension point	After step 3: overdue book



How do we represent it in the use case description?

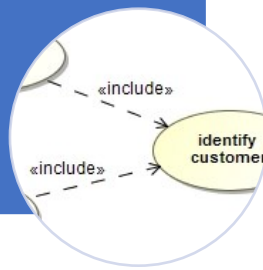
Use case Name	Issue fine
Id	UC19
Description	The librarian records and prints out a fine
Primary actors	Librarian
Secondary actors	none
Preconditions	The returned book is overdue
Main flow	1. The librarian enters details of the fine into the system 2. The system prints out the fine
Postconditions	1. The fine has been recorded in the system 2. The system has printed out the fine
Alternative flows	none



Include versus extend

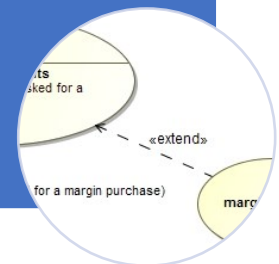
- Used to extract use case fragments that are duplicated in multiple use cases
- The included case cannot stand alone and the original use case is not complete without the included one

Include



- Used to **conditionally** add steps to another class use case
- The basic use case can stand on its own, without the extension

Extend



Include versus extend

	Include use case	Extend use case
Is the included/extended use case optional?	No	Yes
Is the base use case complete without this use case	No	Yes
Is the execution of the included/extended use case conditional?	No	Yes
Does this included/extended use case change the behaviour of the base use case?	No	Yes

[Source: Robert Maksimchuk & Eric Naiburg: *UML for Mere Mortals*, Addison-Wesley, 2005.]

What errors?

Use case name is
not in the oval



Unclear boundaries

The process

Extract the functional requirements (use cases)



Identify actors that are going to interact with the functionalities of the system



Define the relationships among functional requirements (specialization, include, extend, ...)

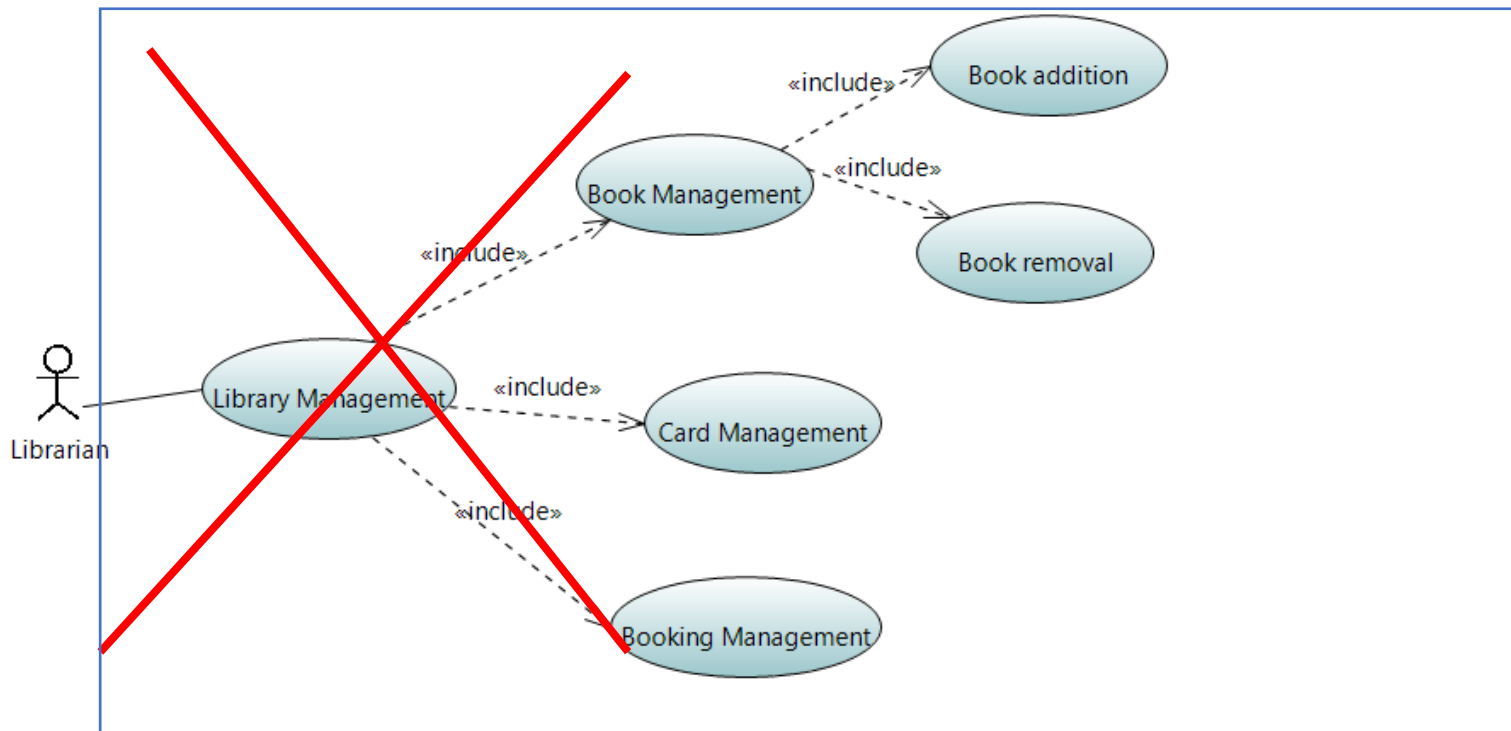


Document in tabular form the use case scenario

Some suggestions

- Find the right level of abstraction for the use cases
 - Neither too abstract nor too detailed
- Do not use the use cases for describing the system internal functionalities (they do not describe interaction!)
- Use extend and generalization only when this helps simplify the model
- Do not mix WHAT with HOW
 - The system asks the customer to confirm the order OK Wrong!
 - The customer presses the OK button NO This is a design choice!
- Avoid functional decomposition
 - Do not create high-level use cases and then split it in lower level use cases

Avoid functional decomposition



High level use cases are extremely abstract and they do not add a lot. Only low levels are meaningful

Few final considerations

- Identifying and designing use cases is complex
 - It seems simple but it is one of the most complicated diagrams in UML
- Use cases should only refer to functional requirements
 - Do not try to transform a non-functional requirement into a use case
- Use the concepts of generalization among use cases and actors
 - Do not try to put all the information in a unique diagram
 - Start from a high-level view with all functionalities and then detailed use cases

Use-case versus actor-centric

- For each use case, all the actors are identified
 - It can become large and complex

Use-case
centric

A diagram showing a light blue oval containing the text "Verify the order", which is a use case, enclosed within a larger white circle.

Verify the order

- For each actor, all the use cases are identified

Actor-based
centric



Customer

Typical errors with use case diagrams

- Use case name that appears multiple times in the diagram
- Arrows between use cases are not dashed (- - - - - >) unless labeled with “extend” or “include”
- «extend»: the arrow goes from the case that describes the event alternative to the standard case
- «include»: the arrow goes from the calling case to the case which describes the actions to be included

Typical errors on the scenarios

- Absence of preconditions
- Failure to connect to the graphical representation
- Exceptional flow: lack of information in the main flow with respect to the point where the condition should be checked

Let's try a test

Join this Wooclap event



1

Go to wooclap.com

2

Enter the event code in the top banner

Event code

SAHUKX

 [Copy participation link](#)



Exercise: magnetic keys

Draw the use case diagram related to the system described in the following description

- For security reasons, an organization has decided to implement a system according to which each employee is assigned by the manager a key card to access (open) certain rooms.
- Access rights will depend on the position and responsibilities of the employee. A manager is in charge to assign the access rights of a key and to change them if its owner changes role in the organization.
- In order to assign a key card the manager has to verify the identity of the employee and their role through the employee db system and then assign the rights related to that role
- In order to update the rights, the manager has to verify the identity of the employee and the new role. At this point the rights can be updated
- When the key is inserted in the door, the rights of the employee are verified. In the positive case, the door is opened, otherwise the door will remain closed, and an alarm is sent to the central station.

[inspired by http://didawiki.di.unipi.it/lib/exe/fetch.php/informatica/is-a/is_05_e1_esercitazionereq_casid_uso.pdf]



Exercise: magnetic keys

3. Identify the functionalities

- The system has to allow to assign a magnetic key to the employee
- The system has to allow to update the access rights of a magnetic key
- The system has to allow to access the rooms with the magnetic key



Exercise: magnetic keys

3.1 search for shared functionalities to model with an include use case

Are there use cases that can be included , i.e., behaviors shared by more than one use case?

- Both the assignment and the update of the key card demand for a verification of the role of the user.
- We can have an included use case Verify

3.2 search for functionalities to be modeled with the extend use case

Include use cases

3.1 search for shared functionalities to model with an include use case

Are

3.2 search for functionalities to be modeled with the extend use case

Summary

- Models
- Modeling languages
- System modelling perspectives
- Use cases and use case diagrams
 - Use case boundaries
 - Actors
 - Use cases and scenarios
 - Actor – use case associations
- Actor generalizations

SUMMARY

